



A structure-based genetic programming generation constructive hyper-heuristic with transfer learning for combinatorial optimisation

Darius Scheepers¹ · Nelishia Pillay¹

Received: 4 November 2024 / Revised: 12 September 2025 / Accepted: 8 December 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Generation constructive hyper-heuristics have proven to be very effective at creating construction heuristics for combinatorial optimization problems, with the heuristics derived by these hyper-heuristics often outperforming human-derived heuristics. Genetic programming has been predominantly used by generation constructive hyper-heuristics. This study investigates the effectiveness of two emerging technologies in genetic programming, namely, transfer learning and structure-based genetic programming, in genetic programming constructive generation hyper-heuristics. The study investigates whether structure-based genetic programming in generation constructive hyper-heuristics with (SBGP-HH-TL) and without transfer learning (SBGP-HH). The hyper-heuristics were evaluated on three problem domains, namely, the examination timetabling problem, the one-dimensional bin packing problem and the capacitated vehicle routing problem. Both SBGP-HH-TL and SBGP-HH outperformed the generational hyper-heuristic employing canonical genetic programming (CGP-HH) on a majority of the problem instances for the three problem domains, with SBGP-HH-TL outperforming SBGP-HH. Hence, the study has revealed that both transfer learning and structure-based genetic programming have resulted in performance improvements in genetic programming generation constructive hyper-heuristics for combinatorial optimization.

Keywords Generation constructive hyper-heuristic · Genetic programming · Structure-based genetic programming · Transfer learning in generation constructive hyper-heuristics

✉ Darius Scheepers
dariusscheepers@gmail.com

Nelishia Pillay
nelishia.pillay@up.ac.za

¹ Department of Computer Science, University of Pretoria, Lynnwood Rd., 0028 Pretoria, Gauteng, South Africa

1 Introduction

Construction heuristics play an important role in metaheuristics by providing a starting point for the search [1]. Prior to the emergence of hyper-heuristics [2], construction heuristics were manually derived. Generation constructive hyper-heuristics are effective at creating construction heuristics for combinatorial optimization problems [2], i.e., to automate the process of creating construction heuristics. Previous work in the field has shown that construction heuristics produced by generation constructive hyper-heuristics have outperformed manually-derived heuristics. [2–4].

Genetic programming [5](GP) is generally used in generation constructive hyper-heuristics to create new construction heuristics. Since the inception of genetic programming generation constructive hyper-heuristics there have been advances in the field of genetic programming. This study investigates whether two such advances, namely, transfer learning [6–8] and structure-based genetic programming [9–11], improves the performance of generation constructive hyper-heuristics for combinatorial optimization.

Transfer learning in genetic programming involves transferring knowledge from a source GP algorithm to a target GP algorithm to improve the performance of the target GP in terms of program quality and/or computational cost. The source GP algorithm is used to solve a simpler problem, while the target GP algorithm a more complicated problem. Knowledge generally transferred includes program trees or subtrees. Previous work has shown that GP with transfer learning performs better than canonical GP [12].

Traditionally, genetic programming uses fitness to direct the search in the program space. Structure-based genetic programming directs the search using both fitness and structure. Research in this area has shown that structure-based genetic programming is able to escape local optima and outperform canonical genetic programming [9–11].

This study compares the performance of canonical genetic programming (CGP-HH), structure-based genetic programming (SBGP-HH) and structure-based genetic programming with transfer learning (SBGP-HH-TL) in generation constructive hyper-heuristics. The performance of these hyper-heuristics were evaluated on three combinatorial optimisation problems, namely, the examination timetabling problem, the one-dimensional bin packing problem and the capacitated vehicle routing problem. These problem domains were selected for evaluating the approaches because:

- The examination timetabling and one-dimensional bin packing problems were the first two problem domains that genetic programming generation constructive hyper-heuristics were applied to, and the heuristics generated by the hyper-heuristics outperformed human-derived construction heuristics. As such, previous genetic programming hyper-heuristics would serve as a benchmark to compare the performance of SBGP-HH and SBGP-HH-TL to.
- It is also important to compare the performance of these algorithms on a problem domain that genetic programming generation constructive hyper-heuristics have not previously been applied to. The capacitated vehicle routing problem has been included for this reason.
- To evaluate the proposed approaches on different problems. All three of these

problems are different discrete optimisation problems that represent real-life scenarios that are being faced in industry.

The study revealed that structure-based genetic programming performed better than canonical genetic programming hyper-heuristics for majority of the problem instances for all three problem domains. Furthermore, the use of transfer learning in both CGP-HH and SBGP-HH resulted in performance improvements. This is the first study to the knowledge of the authors using structure-based genetic programming and structure-based genetic programming with transfer learning to improve the performance of generation constructive hyper-heuristics. Thus, the main contributions of this research are:

- An improvement in performance of genetic programming generation constructive hyper-heuristics for discrete optimisation using structure-based genetic programming.
- An improvement in the performance of genetic programming generation constructive hyper-heuristics using structure-based genetic programming and transfer learning.

The following section provides an overview of the related work, namely, constructive hyper-heuristics, transfer learning, structure-based genetic programming and the three problem domains used in this study to evaluate the performance of the proposed approaches. The canonical genetic programming algorithm as well as transfer learning in CGP-HH are presented in 3. Section 4 describes the structure-based genetic programming algorithm used in this study and transfer learning in SBGP-HH. The experimental setup used to evaluate the approaches is presented in section 5. Section 6 discusses and compares the performance of CGP-HH, SBGP-HH and SBGP-HH-TL. Finally, a summary of the findings of the study and future work is presented in section 7.

2 Related work

This section provides an overview of generation constructive hyper-heuristics, transfer learning and structure-based genetic programming. Section 2.1 describes constructive hyper-heuristics. Section 3.1 introduces transfer learning in genetic programming and provides a brief survey of transfer learning in genetic programming. Section 2.3 introduces structure-based genetic programming and provides an overview of previous studies using structure-based GP. The section then describes each of the problem domains, namely, the examination timetabling problem (section 2.4), the one dimensional bin packing problem (section 2.5) and the capacitated vehicle routing problem(section 2.6). For each of the problems an overview of the problem is firstly given. The construction heuristics specific to the problem are then described. Finally, state of the art techniques for each of the problems are presented.

2.1 Hyper-heuristics

Essentially when solving a combinatorial optimisation problem the solution space is explored. Hyper-heuristics explore a heuristic space instead of a solution space [2]. The heuristic space is mapped to the solution space. The space explored by a hyper-heuristic can contain constructive heuristics for creating a solution or initial point in the search space or perturbative heuristics for improving an initial point. For example, in using a genetic algorithm to solve a routing problem exploring the solution space, each element of the population will be a route. However, a hyper-heuristic will explore the space of construction or perturbative heuristics to find the combination of heuristics or heuristic to create a new solution (construction) or to improve an existing solution (perturbative). As this paper focuses on generation constructive hyper-heuristics, we provide an overview of constructive hyper-heuristics. Constructive hyper-heuristics select or create constructive heuristics that create an initial solution/point in the solution space by exploring the heuristic space [2, 13]. Selection constructive hyper-heuristics select the constructive heuristic to be used at each point when creating a solution. These construction heuristics are human-derived. Generation constructive hyper-heuristics create new constructive heuristics.

Generation constructive hyper-heuristics combine problem-specific attributes with arithmetic and or selection operators to create construction heuristics [3, 4, 14–17]. Genetic programming has predominately been used for this purpose in generation constructive hyper-heuristics. The examination timetabling problem and the one-dimensional bin packing problem are the two domains that generation constructive hyper-heuristics have proven to be very effective for, outperforming human-derived heuristics [2].

While this study focuses on generation constructive hyper-heuristics, we compare the performance of the proposed hyper-heuristics to selection constructive hyper-heuristics as well generation constructive hyper-heuristics in section 6.4. Selection constructive hyper-heuristics select the construction heuristics to apply in order to create a solution.

2.2 Transfer learning in genetic programming generation hyper-heuristics

Transfer learning in neural networks involves freezing certain layers in the neural network and only training certain layers [18]. Transfer learning has proven to be effective in neural networks, resulting in performance improvements, a reduction in training time and a reduction in the amount of data needed. This has led to researchers examining the use of transfer learning in evolutionary algorithms including genetic programming. Genetic programming with transfer learning has been used for image classification [19], evolving policy rules [7], feature or data transfer [20, 21] and symbolic regression [12, 22, 23]. We restrict our survey of previous work to symbolic regression as this application is relevant to this study.

Transfer learning in generation constructive hyper-heuristics employing genetic programming involves the same process as transfer learning in genetic programming for symbolic regression [13], hence we provide an overview of studies involving both below. Transfer learning in genetic programming involves transferring knowl-

edge from a source genetic programming algorithm to a target genetic programming algorithm [12, 24]. The source genetic programming algorithm is usually applied to a simpler problem with the aim of extracting knowledge that can be used by the target genetic programming algorithm in solving a more complicated problem with the aim of producing better solutions and/or reducing computational cost by transferring the knowledge. Transfer learning has resulted in the improvement of the quality of solution programs produced, as well as a reduction in computational cost.

There have been various studies on the use of transfer of learning in genetic programming, these generally differ in terms of the knowledge transferred. In earlier work elements of the population, i.e., syntax trees, were transferred as is or as a set of subtrees from the source genetic programming algorithms to the target genetic programming algorithm [6, 8, 12, 22]. The knowledge is usually used in creating the initial population of the target genetic programming algorithm. These trees and subtrees represent points in the search space. Scheepers and Pillay [6] show the effectiveness of transferring areas of the search space instead of points. Mei et al. [8] transferred elements of the terminal set and the function set. More recently, Liu et al. [25] introduced the concept of transferring mutual information between source and target genetic programming algorithms. A mutual information extractor is used to identify subtrees to transfer from the source genetic programming algorithm to the target genetic programming algorithm. The extractor calculates a probability of selecting a subtree based on the target data and primitives in common with the target genetic programming algorithm. In all these studies GP with transfer learning outperformed canonical GP and in some instances produced results competitive with state-of-the-art approaches and lower computational cost.

The potential of using transfer learning in genetic programming and hence generation constructive hyper-heuristics is clear. Thus, we include transfer learning in the structure-based genetic programming hyper-heuristic in this study and evaluate it further.

2.3 Structure-based genetic programming

Fitness directs the search in genetic programming. Structure-based genetic programming was introduced in [9] to escape local optima in canonical genetic programming. Structure-based genetic programming uses structure and fitness to direct the search. Structure is used to direct the search to perform exploitation (local level search) in a particular area or to perform exploration (global level search) if a local optimum has been reached. When performing local level search a similarity index is used to ensure the search is performed in a specific area, i.e., all the elements of the population are the same for n levels of the tree. Similarly, when performing a global level search a similarity index is used to ensure that areas that previously led to a local optimum are not revisited. Structure-based genetic programming was used to find solutions to complex problems in automatic programming which canonical genetic programming could not solve.

Kapoor and Pillay [10, 11] compared the performance of canonical genetic programming and structure-based genetic programming for neural architecture search for image classification and video shorts creation. Structure-based genetic program-

ming outperformed canonical genetic programming and genetic algorithms for neural architecture search for both image classification and video shorts creation.

Structure-based genetic programming has not been used in generation hyper-heuristics. It is also a fairly new variation of genetic programming with not much research into its use, except for the study described above. Given that it has outperformed canonical genetic programming in all applications thus far, we hypothesise that it will provide an improvement in performance over canonical genetic programming generation construction hyper-heuristics.

2.4 Examination timetabling problem (ETP)

2.4.1 Overview

The examination timetabling problem involves the scheduling of exams in a timetable. The goal of the ETP is to schedule all exams in the timetable while adhering to the hard constraints and minimising the number of soft constraints violated. All hard constraints must be met in order for a timetable to be feasible. Thus, a mathematical formulation of the ETP, where n is the number of hard constraints and m is the number of soft constraints, is:

$$\sum_{i=1}^n hc_i = 0$$

$$\min \sum_{j=1}^m sc_j$$

Examples of hard constraint include there should be no clashes, i.e. no student must be scheduled to write more than one examination at the same time; the room capacity must be sufficient for the number of students writing the examination. Examples of soft constraints include examinations must be well spaced for students; examinations with large numbers should be scheduled early in the timetable. The hard and soft constraints differ for each dataset.

2.4.2 Constructive hyper-heuristics

There are two types of constructive hyper-heuristics, namely, selection constructive and generation constructive. This study focuses on generation constructive hyper-heuristics, however we also compare the performance of the hyper-heuristics in this study to selection constructive hyper-heuristics in section 6.4 thus this section provides and overview of previous work using both these constructive hyper-heuristics for the ETP.

Various selection constructive hyper-heuristics have been applied to the ITC 2007 examination timetabling dataset. Pillay [26] employed a genetic algorithm selection constructive hyper-heuristic for the examination timetabling problem. Each chromosome was a combination of construction heuristics that were applied in sequence to

create an initial timetable. The selection hyper-heuristics performed better than the human-derived construction heuristics.

The selection constructive hyper-heuristic employed by Sabar et al. [27] combine four construction heuristics, namely, largest degree, saturation degree, largest coloured degree and largest enrolment, hierarchically. The hyper-heuristic produced competitive results outperforming some of the state of the art techniques for the ITC 2007 benchmark set.

Soghier and Qu [28] used bin packing construction heuristics to select slots and rooms to all allocate examinations in to create timetables. The selection hyper-heuristic makes a random initial selection of the construction heuristics which is then iteratively improved. The hyper-heuristics produced competitive results.

Rahman et al. [29] proposed an approach that selected combinations of heuristics based on the difficulty of assigning exams. Thereafter, the exams were sorted in descending order based on this difficulty score and each inserted sequentially into a slot that would cause the smallest soft constraint cost increase. This approach performed well on the ITC 2007 dataset.

Generation constructive hyperheuristics have not been as well researched as selection constructive hyperheuristics. In the study conducted by Pillay and Ozcan [3] a genetic programming generation constructive hyper-heuristic was used to create arithmetic construction heuristics for the examination timetabling problem. The evolved heuristics produced better results than the human-derived heuristics, namely, largest degree, saturation degree, largest weighted degree, largest colour degree and largest enrolment, for the ITC 2007 benchmark problem set and also outperformed the hierarchical heuristics produced by a genetic algorithm for the problem set.

2.4.3 State of the art

This section provides an overview of the state of the art approaches applied to the ITC 2007 benchmark set. The performance of the hyper-heuristics proposed in this study are compared to these approaches in section 6.5.

Muller [30] employed a three-phase algorithm to solve the ETP. First, an iterative local search was used to construct a feasible solution. Second, hill climbing was applied to the initial solution to reduce soft constraint cost. Finally, the great deluge algorithm was applied. This approach was the winner for the ITC 2007 competition.

Atsuta et al. [31] employed a single-point search which iteratively searched locally from an initial solution point, marking local optima as tabu. The approach proved to be effective and placed third in the ITC 2007 competition.

Gogos et al. [32] presented a three-phase algorithm that achieved second place in the ITC 2007 exam track. First, an initial solution was created using the saturation degree construction heuristic. Iterative local search was applied to the initial timetable to achieve feasibility. Finally, the great deluge search was used to minimise the soft constraint cost.

In the study conducted by Bykov and Petrovic [33] the initial solution was also constructed using a saturation degree heuristic to select the the period while the room was randomly selected. Late acceptance hill-climbing was used to improve the ini-

tial solution. The approach produced best results for some of the ITC 2007 problem instances.

Rajah [34] proposed a multi-point search meta-heuristic algorithm for ETP to search the space of partial solutions. An initial solution was created using the saturation degree heuristic. Partial solutions were grouped into areas based on the timetable structure. The search focused on optimising within groups of partial solutions during construction until a complete solution was found. Perturbative heuristics were applied to perform exploitation between partial solutions within the same search space. The approach produced best results for some of the ITC 2007 problem instances.

Van Bulk et al. [35] employed multi-neighbourhood simulated annealing to solve the ITC-2007 examination timetabling problem. The study compares performing 1, 10 and a 100 runs of the proposed approach. Performing a 100 runs produced the best performance producing the best results for seven of the problem instances.

2.5 One-dimensional bin packing problem (ODBPP)

2.5.1 Overview

The ODBPP presents a problem where items of variable sizes are to be assigned to bins of fixed capacity so as to minimise the amount of bins required for all items to be assigned. This problem is an NP-hard problem [36].

For each ODBPP problem instance the following specifications are define:

- List of items with specified sizes.
- The capacity of the bins.
- All items are at most as large as a bin.
- As many bins as necessary may be used for all items to be assigned.
- Each item is to be assigned to exactly one bin.
- A bin may contain more than one item, but may not contain a combination of items in which their total size exceeds the capacity of the bin.

Violating any of the last two constraints would result in a infeasible solution. The objective is to minimise the number of bins that contain at least one item each. Thus, a mathematical formulation of the problem with n the number of bins, i representing the i th bin in the range $1...m$, o_j is the j th item in bin b_i and c_i the capacity of the i th bin is:

$$\min \sum_{i=1}^n b_i$$

where for each i :

$$\sum_{j=1}^m o_j \leq c_i$$

2.5.2 Constructive hyper-heuristics

This section provides an overview of the selection and generation and construction hyper-heuristics that the generation hyper-heuristics are compared to in section 6.4.

In the selection constructive hyper-heuristic employed by Ross et al. [37] construction heuristics are selected based on the state of the art solution under construction. A messy genetic algorithm was used to drive the search. This approach provided the means of intelligently selecting construction heuristics based on the state of the problems. The hyper-heuristic outperformed the human-derived construction heuristics.

Pillay [38] presented two selection constructive hyper-heuristics employing genetic algorithms. One was used to select bins for receiving items and the other selected both the bin and the item to be assigned to the bin. Both hyper-heuristics outperformed the human-derived heuristics with the second hyper-heuristic outperforming the first.

The first application of generation constructive hyper-heuristics for combinatorial optimisation was for the one-dimensional bin packing problem. Burke and Hyde [4, 16, 17] examined the use of genetic programming generation construction hyper-heuristics for the one-dimensional, two-dimensional, three-dimensional and knapsack packing problems. The evolved heuristics outperformed human-derived heuristics and produced results comparative to that of state of the art approaches.

Sim and Hart [14] employ a single-node genetic programming generation constructive hyper-heuristic for the ODBPP. An island model is used to search in different areas of the heuristic space simultaneously. Each island produced heuristics capable of solving more than one problem instance of ODBPP optimally. A list of heuristics was maintained to solve any new unseen problem instance. The approach outperformed previous constructive hyper-heuristics.

A generation construction hyper-heuristic inspired by human immune systems was employed by Sim et al. [39]. The proposed system maintained a population of generated heuristics in which each heuristic produced a better result than the other heuristics for one or more problem instances. In a later work [40] this approach was extended to include an improved mutation operator. The approach outperformed previous constructive hyper-heuristic approaches.

Singh and Pillay [41] present an ant colony optimisation generation constructive hyper-heuristic. Pheromone maps of different dimensions were evaluated in the study. Three dimensional maps produced the best results. The hyper-heuristic outperformed human-derived heuristics.

2.5.3 State of the art

A metaheuristic to construct and optimise a solution for the ODBPP was proposed by Alvim et al. [42]. The algorithm comprised the following phases: constructing an initial solution using bin selection heuristics, distributing the load according to the dominance of the items, balancing the load between the bins and finally fixing any infeasibilities. The hyper-heuristic produced good results for the Faulkenauer dataset.

Fleszar and Charalambous [43] proposed a bin selection algorithm that took into account the average weight of packed items. Reduction and perturbative methods

were used to further improve an initial solution created using the minimum slack heuristic. The approach achieved optimal values for the Scholl datasets.

Dokeroglu and Cosar [44] presented a grouping genetic algorithm and bin selection heuristics to obtain solutions for ODBPP datasets. The approach was applied to a number of datasets including the Scholl dataset. The approach solved 90% of the instances to optimality.

Quiroz-Castellanos et al. [45] presented a grouping genetic algorithm that explored the solution space. The approach controlled the balance between diversity (exploration) and selective pressure (exploitation), thus reducing the chances of premature convergence. The approach solved all instances of the Scholl dataset to optimality.

Kucukyilmaz and Kiziloz [46] proposed a hybrid of an island model and a grouping genetic model. The algorithm was scalable and could be parallelised to improve computation cost. The approach solved all instances of the Scholl dataset to optimality.

Silva and Borges [47] proposed a hybrid metaheuristics, including a genetic algorithm, to solve the ODBPP. The hybrid model was used to avoid getting stuck in local optima as a result of using the genetic algorithm. From the results, it was shown that optimal values were achieved for nearly all instances of the Falkenauer dataset

2.6 Capacitated vehicle routing problem (CVRP)

2.6.1 Overview

This problem domain is an extension of the famous, NP-hard, travelling salesman problem. The goal is to minimise the total distance covered to visit customers with different geographic locations while adhering to hard constraints.

For each CVRP problem instance the following is specified:

- A depot and a set of customers exist each of which contain two-dimensional coordinates.
- Customers also have a demand value and a service time.
- The service time is the same for all customers.
- Vehicles with a fixed capacity and maximum cost
- A customer can only be serviced by exactly one vehicle.
- Each vehicle needs to start and end its journeys at the depot.
- The vehicles can service as many customers as long as the cumulative demand of its selected customers does not exceed its capacity.
- Each vehicle has its own route containing the customers that it will service.
- The cost of a vehicle is calculated as the total distance covered by the vehicle plus the service time spent at each customer. The cost of each vehicle may not exceed a predefined cost.

The objective is to minimise the total distance covered by all vehicles over all routes. Mathematically, this can be expressed as:

$$\min \sum_{i=1}^m R_i$$

where each R_i is:

$$R_i = \sum_{j=1}^n c_{ji} + \sum_{j=1}^n s_j$$

The objective value to be minimized is the sum of the cost of m routes R_i . The cost of each route is the sum of the cost c_{ji} of travelling between two customers for n customers plus the sum of the service fee for each customer.

There is no limit to the number of vehicles that can be used to solve the problem.

2.6.2 State of the art

Taillard [48] proposed an approach that iteratively inserts selected customers into routes while keeping a memory for tabu routes. This approach also decomposed a given problem instance into sub-problems according to sectors around the depot, which allowed solving these sub-problems in parallel. Furthermore, routes were individually optimised using an algorithm by Volgenant and Jonker [49]. Optimal solutions were found for some of the problem instances.

Rochat and Taillard [50] proposed a local search method that used adaptive memory. This approach used a tournament of quality solutions that could be added or removed accordingly. Best known solutions were found for two instances of the Christofides dataset.

Xu and Kelly [51] presented the solution to the problem as a graph with different partial solutions as vertices and applied a tabu search for optimisation. Route optimisations were also performed using an algorithm by Lin [52]. The results were comparable to state of the art approaches.

Toth and Vigo [53] proposed a granular tabu search algorithm. The algorithm was designed to eliminate long edges between customers within the routes. The algorithms iteratively swaps routes between neighbouring solutions where the distances exceeded a threshold. This threshold gradually allowed for more solutions when improvements could not be found. Some solutions were shown to match those of state of the art approaches.

3 Canonical genetic programming hyper-heuristic

This section describes the canonical GP algorithm. The algorithm CGP-HH is firstly presented. The section then describes transfer learning used in the algorithm CGP-HH-TL.

The generational GP algorithm [5] depicted in Algorithm [5] is used in this study. The algorithm iteratively refines an initial population over a number of generations.

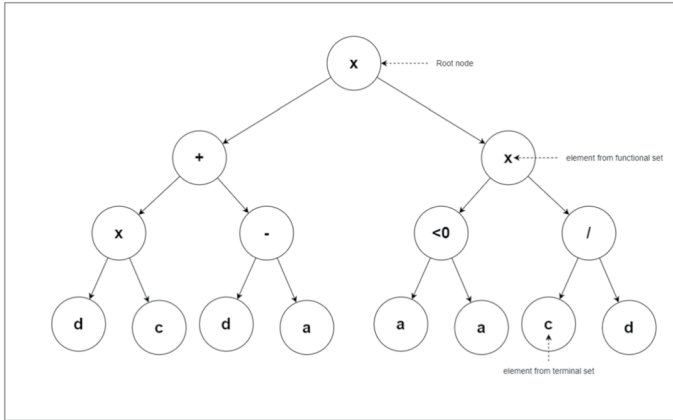


Fig. 1 Example of an element of the population

Table 1 Function Set

Element	Description
Plus (+)	Performs addition
Minus (-)	Performs subtraction
Product ($\times$)	Performs multiplication
Protected division (/)	Performs protected division which returns the numerator if the denominator is zero
Minimum (min)	Returns the minimum of the two operands
If Less than Zero (< 0)	Takes three arguments, if the first argument is less than zero it returns the second argument otherwise it returns the first argument

Representation, initial population generation, evaluation and selection, genetic operators and termination criteria are described in the sections that follow.

Each element of the population is a syntax tree representing a construction heuristic. An example is depicted in 1.

Each heuristic is comprised of arithmetic operators and problem specific attributes. The arithmetic operators used in this study are listed in Table 1. The terminal set is comprised of problem attributes, e.g., number of examinations, bin capacity and/or human-derived construction heuristics. The terminal set used for each problem domain is described section 5.

Each element of the population is created randomly using the grow method [5]. Duplicate trees are not permitted in the initial population.

Each element of the population is evaluated by using it to construct an initial solution to the problem at hand and the quality of the solution produced is the fitness. The fitness function used for each of the three problem domains is presented in section 5.

Tournament selection [5] is used to select parents. If two elements of the tournament have the same fitness, the element with smaller number of nodes in the tree is selected.

The crossover and mutation operators [5] are used to create the offspring of each generation. The offspring produced must not exceed the maximum depth permitted.

Two termination criteria are used for all GP approaches:

- Convergence - an algorithm is marked as converged if the standard deviation over the last thirty generations is zero.
- A maximum number of generations is reached.

3.1 Transfer learning

Transfer learning involves transferring knowledge from a source GP algorithm to a target GP algorithm. The GP algorithm described above is used for both the source GP and the target GP. The CGP-HH is used to solve q source problem instances independently. The best elements from each of the runs form part of the initial population of the target GP, CGP-HH-TL, for each target problem instance. The remainder of the initial population is created randomly using the grow method [5]. It is anticipated that knowledge transferred will have impact throughout a genetic programming run, thus evaluation of transfer learning is evaluated over a run.

4 Structure-based genetic programming hyper-heuristic

This section describes the structure-based genetic programming hyper-heuristic SBGP-HH. The SBGP-HH is depicted in Algorithm 1.

```

1: Create an initial population
2: while no stopping criteria have been met do
3:   Global level search
4:   Local level search
5: end while
6: return tree with best fitness

```

Algorithm 1 SBGP-HH Algorithm

After initial population generation the algorithm iterates between a global level search that performs exploration and a local level search that performs exploitation. The first global level search runs the canonical GP algorithm for S_g generations. A global area is extract from the best individual found over the S_g generations. The global area is the first D_g of the best individual. Both S_g and D_g are parameters. An example of a global area is depicted in figure 2. In this case the global area is denoted by the first three layers of the search space.

The local level search is performed for W_g generations to exploit the global area. The first n layers of all the elements of the populations for these generations are fixed to be that of the global area identified. The genetic operators are applied from level $n + 1$ onwards. The local level search is run until a point of convergence is reached, i.e., when there has been no change in fitness for the last last W_g generations. This process of performing the global level search and local level search in succession is repeated until one of the termination criterion of the CGP-HH are met.

The second and subsequent global level search differs from the first global level search. In the case of the former, evolution is applied to the first D_g levels of the ele-

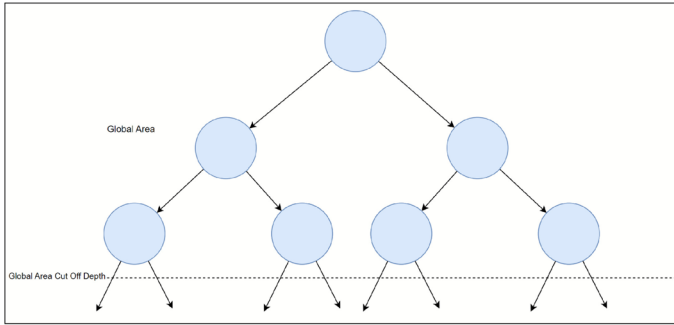


Fig. 2 Example Global Area

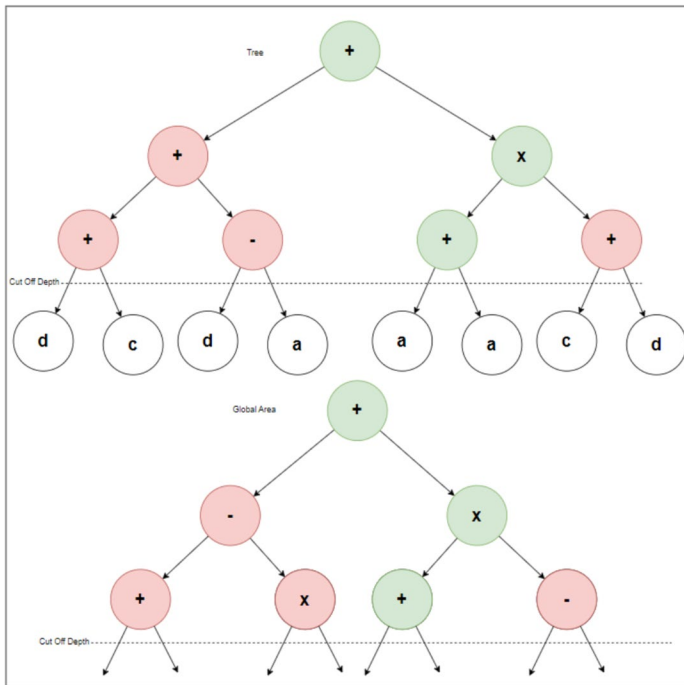


Fig. 3 Similarity Index Example

ments of the population for S_g generations. The aim is to evolve these D_g levels to be different from the global areas identified in previous iterations. This is achieved by ensuring that the D_g levels of the offspring are sufficiently different from the global areas of the previous iterations. A similarity index is calculated for each offspring and previous global areas. The value of this index must be less than the global similarity index threshold T_g , which is a parameter, otherwise the offspring is discarded.

An example of how the similarity index is calculated is depicted in figure 3. In this example D_g is three. The similarity index is calculated by counting the number of

nodes that are in the same position in order in both trees. These are indicated in green in the example. In this example the similarity index is 3.

4.1 Transfer learning

Transfer learning in CGP-HH-TL involves transferring the best performing elements from the source GP to the initial population of the target GP. The transfer learning in SBGP-HH-TL is similar with the difference being in the knowledge being transferred. Instead of transferring the best performing elements to the initial population of the target GPs, the global areas are transferred. Thus, SBGP-HH-TL employs the same algorithm depicted in Algorithm 1 with the initial population created using the transferred global areas. The source SBGP-HH runs on q source problem instances for u runs. The best trees are selected from all the source SBGP-HH runs. From each tree, a global area is extracted, which is the first D_g levels of a tree in the target initial population. For each global area, τ number of trees are created with this global area structure, and the rest of the tree from level $D_g + 1$ is created using the grow method [54] to a depth of $D_{I_g} - D_g$. The value τ is calculated as in equation 1, where P is the population size for the target GP.

$$\tau = \text{round}(P/(q * u)) \quad (1)$$

5 Experimental setup

This section describes the experimental setup used to evaluate the proposed approaches. Section 5.1 provides an overview of the datasets used for each of the three problem domains. The performance metrics and statistical tests used in the study are defined in section 5.2 and section 5.3 respectively. The process used to tune the parameters is presented in section 5.4. Section 5.5 lists the technical specifications of the hardware and software used in this study.

5.1 Datasets

This section describes the details for the datasets used for each of the problem domains. The following is provided for each dataset:

- a description of the dataset
- the terminal set used by the GP algorithm
- the source and target problems instances for transfer learning

5.1.1 Examination timetabling problem

The ITC 2007 [55] benchmark set was used in this study. It contains 12 instances. The details of each instance are listed in Table 2.

Table 2 ITC 2007 dataset instances' information

Instance	Exams	Students	Periods	Rooms	Conflict Density (%)
ETP1	607	7891	54	7	5.05
ETP2	870	12743	40	49	1.17
ETP3	934	16439	36	48	2.62
ETP4	873	5045	21	1	15
ETP5	1018	9253	42	3	0.87
ETP6	242	7909	16	8	6.16
ETP7	1096	14676	80	15	1.93
ETP8	598	7718	45	8	4.55
ETP9	169	655	25	3	0.08
ETP10	214	1577	32	4	0.05
ETP11	934	16439	26	40	0.03
ETP12	78	1653	12	50	0.18

The conflict density is the potential number of enrolled students that can be involved in clashes over the total number of enrolled students. The larger the value, the more difficult it is to solve the problem. The hard constraints are:

- There are no exam conflicts.
- The total number of students for all exams scheduled in a period must not exceed the capacity of the room.
- The duration of an exam must not exceed that of the period it is scheduled in.
- Requirements regarding the order in which exams are to be scheduled, e.g., a particular exam must take place before or after another exam.

The soft constraints are:

- Two exams in a row - two examinations must be written sequentially on the same day.
- Two exams in a day - two examinations must be written on the same day.
- Spread of examinations - examinations that share students must be a specified number of periods apart.
- Frontload - Examinations with a larger number of students should be placed earlier in the timetable.
- Mixed durations - Examinations scheduled in the same room should have the same duration.
- Period penalty - There is a penalty associated with using certain periods.
- Room penalty - There is a penalty associated with using certain rooms.

A solution to this problem is a feasible timetable, i.e., a timetable with all examinations scheduled and no hard constraint violations. The soft constraint cost is minimised. The fitness function used is a function of the hard and soft constraint cost used for the examination timetabling track for the ITC2007 competition [30].

The terminal set is an extension of that used by Pillay and Ozcan [3]:

- l_a -ETP - Number of potential clashes for the exam.
- l_b -ETP - Number of students involved in potential clashes for the exam.
- l_c -ETP - Number of students taking the exam.
- l_d -ETP - Number of periods in the timetable in which the exam cannot cause hard constraint violations.
- l_e -ETP - Number of potential clashes between this exam and other remaining exams.
- l_f -ETP - The sum of the weighted distances, with a maximum distance of 5, between slots that have students in common.
- l_g -ETP - Number of rooms with the required capacity for the exam.
- l_h -ETP - Number of periods with the required duration for the exam.
- l_i -ETP - Number of periods.
- l_j -ETP - Number of rooms.
- l_k -ETP - Number of exams unscheduled.
- l_l -ETP - Number of enrolments remaining.
- l_m -ETP - Number of exams.

Three of the simpler problem instances, namely, ETP1, ETP2 and ETP8, were used as source problem instances. These were selected based on the study in [3] in which feasible solutions were found using the saturation degree heuristic and did not require further optimisation to attain feasibility. The remaining instances are used as target instances.

5.1.2 One-dimensional bin packing problem

Problem instances from the Scholl [56] and Falkenauer [57] datasets were used:

- Scholl-Easy - 720 problem instances
- Scholl-Medium - 480 problem instances
- Scholl-Hard - 10 problem instances
- Falkenauer_u - 80 problem instances
- Falkenauer_t - 80 problem instances

The objective value for this problem is the number of bins. The objective function is often the same for potential solutions, thus the objective value is not suitable to use as a fitness function. Hence, the fitness function used is the formula proposed by Falkenauer to calculate the amount of wasted space specified in Formula 2:

$$f = 1 - \frac{\sum_{i=1}^n \left(\frac{\sum_{j=1}^m w_j x_{ij}}{C} \right)^2}{n} \quad (2)$$

Where n = number of bins, m = number of items, w_j = weight of item j , C = bin capacity, $x_{ij} = 1$ if item j is in bin i and 0 otherwise. f will produce a value between 0 and 1. The closer this value is to 0, the better the quality of the solution.

The following list contains the terminal set used for the ODBPP datasets, as extended by the work of Sim and Hart [14]:

- l_a -ODBPP - The largest item is packed into the current bin. If the operation is successful a 1 is returned otherwise a -1 is returned.
- l_b -ODBPP - Finds the largest combination of two items and attempts to pack it into the current bin. If the operation is successful a 1 is returned otherwise a -1 is returned.
- l_c -ODBPP - Finds the largest combination of at most two items and attempts to pack it into the current bin. If the operation is successful a 1 is returned otherwise a -1 is returned.
- l_d -ODBPP - Finds the largest combination of at most three items and attempts to pack it into the current bin. If the operation is successful a 1 is returned otherwise a -1 is returned.
- l_e -ODBPP - Finds the largest combination of at most five items and attempts to pack it into the current bin. If the operation is successful a 1 is returned otherwise a -1 is returned.
- l_f -ODBPP - Return s the amount of free space in the current bin.
- l_g -ODBPP - Packs the smallest item in the current bin. Returns 1 if the operation is successful and -1 otherwise.
- l_h -ODBPP - Return s the average fullness of all bins.
- l_i -ODBPP - Return s the number of unpacked items.
- l_j -ODBPP - Return s the number of unpacked items making up two thirds of the items to be packed.
- l_k -ODBPP - Return s the amount of items in the range from one third and less than two thirds of the items to be packed.
- l_m -ODBPP - Return the amount of items less than a third of the items to be packed.

Instances in the Scholl-Easy and Scholl-Medium datasets were the source instances. Twelve problem instances were randomly selected from the combined 1200 problem instances to be source instances. The target instances are the 1358 instances from the remaining datasets.

5.1.3 Capacitated vehicle routing problem

The Christofides [58] dataset has 14 instances. Table 3 shows the characteristics of each instance.

The objective value is the total distance covered by all vehicles for the provided instance. The fitness function is the total distance by all vehicles multiplied by the additional number of vehicles used.

The terminal set used is:

- l_a -CVRP - Demand of the customer.
- l_b -CVRP - Distance from the customer to the depot.
- l_c -CVRP - Number of nodes which are in circular area where radius is a quarter

Table 3 Details of CVRP Christofides dataset instances

Instances	Vehicles	Capacity	Customers	Max Cost	Service time
CVRP1	5	160	51	inf	0
CVRP2	10	140	76	inf	0
CVRP3	8	200	101	inf	0
CVRP4	12	200	151	inf	0
CVRP5	17	200	200	inf	0
CVRP6	6	160	51	200	10
CVRP7	11	140	76	160	10
CVRP8	9	160	101	230	10
CVRP9	14	200	151	200	10
CVRP10	18	200	200	200	10
CVRP11	7	200	121	inf	0
CVRP12	10	200	101	inf	0
CVRP13	11	200	121	720	50
CVRP14	11	200	101	1040	90

of the 2 farthest nodes.

- l_d -CVRP - Average distance from the customer to all other nodes.
- l_e -CVRP - Distance from the current stop/node in the route to the customer.
- l_f -CVRP - Distance from the current stop/node in the route to the customer to the depot.
- l_g -CVRP - Number nodes that have not been visited.
- l_h -CVRP - Average load across all vehicles.
- l_i -CVRP - Difference between the current distance covered and the lowest distance covered thus far.

Table 3 lists the attributes of each problem instance in the Christofides dataset. The less customers a problem instance contains, the less options are to be considered within each step of constructing a solution, thus decreasing runtime and complexity. CVRP1, CVRP2, CVRP5 and CVRP6 contain the smallest amounts of customers and are therefore used as source problem instances. The rest of the problem instances were selected as the target problem instances.

5.2 Performance metrics

This section describes the performance metrics used to assess the quality of solutions produced by CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL.

5.2.1 Objective value

The objective value quantifies the quality of a solution and is used to compare the solutions constructed by the approaches. The calculations of objective values for each problem domain are described in Sect. 5.1. For a provided instance, the approach that produces a solution with the lowest objective value is the best approach. These values

were also used for comparison with human-derived construction heuristics, constructive hyper-heuristics and state-of-the-art algorithms.

5.2.2 Generality

Pillay and Qu [59] introduced a metric to evaluate the generality of an approach across datasets by comparing the performance of the approach for each instance with the best-known result for the instance. This metric, referred to as the standard deviation of the differences (*SDD*), is computed according to formula 3.

$$SDD(A) = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}} \quad (3)$$

$$x_i = \frac{|o_i - b_i|}{\frac{o_i + b_i}{2}} * 100 \quad (4)$$

SDD is the standard deviation over all differences between the best found solutions for an algorithm *A* for each instance and the respective value from the best known solution. x_i is calculated according to Eq. 4, where o_i is the best objective value produced by the algorithm for instance *i* and b_i is the best known value. The lower the *SDD* value, the better *A* generalises.

5.2.3 Computational effort

The computational effort, as defined by Koza [5], measures the number of individuals that are evaluated to find a solution program as defined in equation 5, where *P* is the population size and *i* is the generation where the solution was found.

$$f(x, P, i) = R(x, P, i) * P * i \quad (5)$$

R is the number of generations necessary to obtain a solution with a 99% chance that it will be found within the target number of runs for that data instance. *R* is calculated according to equation 6, where *x* is the certainty that a solution will be found by generation *i* and $B(P, i)$ is the cumulative probability of finding a solution by generation *i*.

$$R(x, P, i) = \lceil \frac{\log(1 - x)}{\log(1 - B(P, i))} \rceil \quad (6)$$

5.3 Statistical tests

Due to the stochastic nature of evolutionary algorithms, thirty runs were performed for each problem. The Mann–WhitneyU test was used to determine the statistical significance of the results when comparing the performance of the proposed algorithms.

A right one-tailed Mann-WhitneyU [60] test is performed with a confidence level of 90% for each comparison. The test involves two hypotheses:

- Null Hypothesis (H_0): There are no differences in the mean objective values of A_a and A_b .
- Alternative Hypothesis (H_a): A_a has a higher mean objective value than A_b .

If the p-value is higher than the critical value of 0.9, then the null hypothesis is rejected and the alternative hypothesis is accepted, which means that the approach A_a has a mean objective value significantly greater than the mean objective value of A_b .

5.4 Parameter tuning

The parameters were tuned in sets for all algorithms, namely, CGP-HH, CGP-HH-TL, SBGP-HH and SBGP-HH-TL, using multiple runs. The initial values for the parameters were taken from previous studies [3, 11]. The set was selected that delivered the best average performance across the runs. The parameters used for all the algorithms are listed in Table 4 with the additional parameters for SBGP-HH and SBGP-HH-TL in Table 5.

5.5 Technical specifications

The algorithms were implemented in Java 1.8 without any external libraries. The simulations were run on a cluster with 24 cores, 2.6 GHz and 128GiB RAM.

6 Results and Discussion

This section discusses and compares the performance of CGP-HH, SBGP-HH and SBGP-HH-TL. Section 6.1 compares the performance of canonical genetic programming and structure-based genetic programming and section 6.2 that of structure-

Table 4 CGP-HH and CGP-HH-TL Parameters

Parameter	Value	Justification
Population Size P	500	Offers adequate initial distribution in genetic material and reasonable runtimes
Crossover / Mutation Application Rate R	0.7 / 0.3	Reasonable balance between reusing and introducing new genetic material
Tournament Size S	4	Offers high probability in not overlooking best individuals and not converging too early
Maximum Tree Depth D_M	8	Adequate room for trees to grow and variation without increase in runtime
Initial Tree Depth D_I	4	Enough for a working tree and enough variations on initialisation
Mutation Maximum Subtree Depth D_U	4	Injects enough new genetic material into a tree

Table 5 SBGP-HH and SBGP-HH-TL Parameters

Parameter	Value	Justification
Number of Generations to Search for Global Area S_g	10	Enough generations to explore to find a global area to exploit
Global Area cutoff Depth D_g	4	The depth at which most similarities between trees are found
Global Similarity Threshold Index T_g	6	Effectively move the search away from exploited areas
Generations Window of Tolerance for no change W_g	10	Enough for the exploitation of a global area to not waste runtime on the area
Initial Tree Depth in Transferred Global Area DI_g	6	Adequate for trees to contain global area with enough distribution in the area

based genetic programming and structure-based genetic programming with transfer learning in generation construction hyper-heuristics. The performance of the structure-based genetic programming hyper-heuristics are analysed in section 6.3. The performance of the hyper-heuristics presented in this paper are then compared to that of human-derived construction heuristics and construction hyper-heuristics (section 6.4). Section 6.5 compares the performance of the hyper-heuristics to state of the art approaches. As the main contribution of this research is not to compare canonical genetic programming with and without transfer learning, this comparison has been included in Appendix A in case the reader is interested in this. Similarly, as a comparison of the performance of canonical genetic programming with transfer learning and structure-based genetic programming with transfer learning is not a contribution of this study, these details are included in Appendix B for completeness. Appendix C contains the p-value tables for the statistical tests conducted.

6.1 Canonical genetic programming(CGP-HH) vs. Structure-Based Genetic Programming(SBGP-HH)

In this section we compare the performance of the canonical genetic programming hyper-heuristic and the structure-based genetic programming hyper-heuristic. Each of the subsections presents the results for each of the problem domains.

6.1.1 Examination timetabling problem

Table 6 lists the minimum objective values (Best), averages (AVG) and standard deviations (STD) of 30 runs of CGP-HH vs. SBGP-HH-TL on the ETP ITC 2007 dataset. From the table, it is evident that SBGP-HH outperforms CGP-HH in 9 out of 12 instances. This result was found to be statistically significant for ETP4, ETP5, ETP6 and ETP9.

Table 7 shows the computational effort values for both approaches. The generation by which a solution program is found is 80. From the table, the computational effort needed by SBGP-HH is less than CGP-HH on 7 instances, including 6 instances where SBGP-HH performs better than CGP-HH as shown in Table 6. This observa-

Table 6 CGP-HH vs SBGP-HH performance on ETP ITC 2007 over 30 runs. Best values are highlighted in bold

Instance	CGP-HH			SBGP-HH		
	Best	AVE	STD	Best	AVE	STD
ETP1	5198	5694.6	220.8	5176	5640.6	219.7
ETP2	478	513.6	20.4	492	528.4	24.3
ETP3	10309	10541.7	177.3	10184	10594.6	245.2
ETP4	17088	32634.8	6641.9	19319	28866.4	8090.3
ETP5	3084	3292.8	98.6	3067	3261.8	112.4
ETP6	26320	26718.2	176.7	26300	26667.2	175.1
ETP7	5206	5242.2	102.9	5065	5267.4	94.9
ETP8	9289	9651.6	185.7	9277	9645	179.8
ETP9	1097	1131.3	19.4	1074	1121.4	28.8
ETP10	14003	14202.2	87	13942	14223.8	121
ETP11	30109	31339.7	668.5	30019	31651.7	954.4
ETP12	5330	5410.3	54.5	5388	5420	69.3

Table 7 CGP-HH vs SBGP-HH Computation Effort for the ETP ITC 2007 dataset. Best values are highlighted in bold

Instance	GP	SBGP
ETP1	281104.24	272680.09
ETP2	133883.94	189392.1
ETP3	312550.96	165788.89
ETP4	121319.24	104690.37
ETP5	231075.77	155687.7
ETP6	128882.75	75740.2
ETP7	184920.66	93596.42
ETP8	232140.14	254464.01
ETP9	78868.05	124413.52
ETP10	200423	268200.75
ETP11	405274.26	182041.66
ETP12	58446.22	96130.34

tion indicates that SBGP-HH is capable of finding the best solution of a run in less generations than CGP-HH for this dataset, which is attributable to the approach of SBGP-HH to exploit specific areas.

The values of the generality CGP-HH and SBGP-HH are 12.107 and 13.642, respectively. These results suggest that CGP-HH demonstrates superior generalisation capabilities compared to SBGP-HH when applied to the ITC 2007 dataset.

6.1.2 One-dimensional bin packing problem

Table 8 provides the number of instances solved to optimum by CGP-HH and SBGP-HH on the ODBPP datasets. From the table, SBGP-HH solved more instances to optimality than CGP-HH for each dataset. This results was found to be statistically significant for 82 problem instances.

Table 9 presents the average computational effort for each dataset. The generation by which a solution program is found is 50. From the table, the computational effort

Table 8 CGP-HH vs SBGP-HH Performance on ODBPP datasets. Best values are highlighted in bold

datasets	CGP-HH			SBGP-HH		
	Opt	Opt.-1	Sum	Opt	Opt.-1	Sum
Scholl Easy (720)	626	90	716	630	82	712
Scholl Medium (480)	380	61	441	380	62	442
Scholl Hard (10)	7	3	10	7	3	10
Falkenauer_u (80)	29	44	73	30	39	69
Falkenauer_t (80)	0	53	53	0	55	55
Total (1370)	1042	251	1293	1047	241	1288

Table 9 CGP-HH vs SBGP-HH average computational effort on ODBPP Datasets. Best values are highlighted in bold

	CGP-HH	SBGP-HH
Scholl Easy	2738.394	4262.864
Scholl Medium	12562.225	23590.508
Scholl Hard	13508.437	24069.791
Falkenauer_u	13939.966	19715.277
Falkenauer_t	11023.883	18986.84

needed by CGP-HH is less than SBGP-HH for each dataset. SBGP-HH finds the best solution of a run later in the algorithm. It is hypothesised that for most instances, SBGP-HH found the best solution of a run not in the first global area it exploited, which shows its effectiveness in escaping a local optimum to explore other global areas.

The values of the generality performance metric for CGP-HH and SBGP-HH 1.218 and 1.203 respectively. These results suggest that SBGP-HH demonstrates superior generalisation capabilities compared to CGP-HH when applied to the ODBPP datasets.

6.1.3 Capacitated vehicle routing problem

Table 10 provides the minimum objective values, averages and standard deviations of 30 runs of CGP-HH vs. SBGP-HH on the CVRP Christofides dataset. From the table it can be seen that SBGP-HH has outperformed CGP-HH in 8 out of 14 instances. This is result was statistically significant for CVRP1, CVRP2, CVRP4, CVRP6 and CVRP7.

Table 11 shows the computational effort values for both approaches. From the table, the computational effort needed by CGP-HH is less than SBGP-HH on 10 instances. This observation indicates that CGP-HH would in most cases find the best solution of a run in earlier generations than SBGP-HH.

The values of the generality performance metric reported for CGP-HH and SBGP-HH are 3.402 and 3.695 respectively. These results indicate that CGP-HH demonstrates superior generalisation capabilities compared to SBGP-HH when applied to the Christofides dataset.

Table 10 CGP-HH vs SBGP-HH performance on CVRP Christofides over 30 runs. Best values are highlighted in bold

Instance	CGP-HH			SBGP-HH		
	Best	AVE	STD	Best	AVE	STD
CVRP1	555.4733	579.8	12.5	550.2551	574.6	11.4
CVRP2	921.2329	950.5	17.6	908.6607	942.2	17.3
CVRP3	906.1137	930.8	13.4	891.7708	925.2	17.1
CVRP4	1149.7097	1210.9	24.4	1164.0816	1208.1	18.5
CVRP5	1453.6618	1524.2	34.1	1449.0727	1519.9	31.5
CVRP6	583.7999	593.7	7.6	571.3939	590	8.2
CVRP7	978.5827	1018.9	11.7	970.2701	1012.9	15.9
CVRP8	952.2692	985.4	25.2	944.3633	980.6	15.4
CVRP9	1295.6049	1344	24.5	1310.8663	1344.9	15.4
CVRP10	1570.4997	1614.1	30.9	1579.271	1612.2	26.2
CVRP11	1082.6853	1133.3	25.1	1109.4358	1156.5	27.3
CVRP12	835.5178	848.9	7	838.2984	852.8	6.4
CVRP13	1598.1916	1615.4	11.8	1599.899	1616.3	9.6
CVRP14	907.9564	936.9	17.7	894.0796	934.8	16

Table 11 CGP-HH vs SBGP-HH Computational Effort for the Christofides dataset. Best values are highlighted in bold

Instance	CGP-HH	SBGP-HH
CVRP1	27148.04	54552.21
CVRP2	15991.03	53203.02
CVRP3	14650	85408.06
CVRP4	20740.62	33666.67
CVRP5	37228.16	50787.04
CVRP6	16400	27639.97
CVRP7	13773.3	49179.51
CVRP8	20823.36	51815.12
CVRP9	46616.21	69840.98
CVRP10	38283.03	38986.28
CVRP11	111686.29	97991.19
CVRP12	63247.83	66109.29
CVRP13	73401.27	42259.26
CVRP14	49710.81	61638.32

6.2 Structure-based genetic programming(SBGP-HH) vs. structure-based genetic programming with transfer learning(SBGP-HH-TL)

This section presents a comparison of the performance of SBGP-HH and SBGP-HH-TL. Each of the following subsections present the results for each problem domain.

6.2.1 Examination timetabling problem

Table 12 provides the minimum objective values, averages and standard deviations of 30 runs for SBGP-HH and SBGP-HH-TL on the ETP ITC 2007 dataset. The source problem instances are not included in the table as the SBGP-HH-TL did not generate heuristics these instances. From the table, it is evident that SBGP-HH-TL out-

Table 12 SBGP-HH vs SBGP-HH-TL performance on ETP ITC 2007 over 30 runs. Best values are highlighted in bold

Instance	SBGP-HH			SBGP-HH-TL		
	Best	AVE	STD	Best	AVE	STD
ETP3	10184	10594.6	245.2	10164	10585.8	208
ETP4	19319	28866.4	8090.3	17821	25247.4	6605.9
ETP5	3067	3261.8	112.4	3083	3247.1	81.1
ETP6	26300	26667.2	175.1	26270	26623.4	184.5
ETP7	5065	5267.4	94.9	5000	5224.6	94.5
ETP9	1074	1121.4	28.8	1071	1110.7	21.1
ETP10	13942	14223.8	121	13954	14184.9	104.2
ETP11	30019	31651.7	954.4	30057	31680.5	1022.4
ETP12	5388	5420	69.3	5339	5424.7	72.8

Table 13 SBGP-HH vs SBGP-HH-TL Computational Effort for the ETP ITC 2007 dataset. Best values are highlighted in bold

Instance	SBGP-HH	SBGP-HH-TL
ETP3	165788.89	106006.55
ETP4	104690.37	280470.95
ETP5	155687.7	222181.57
ETP6	75740.2	103692.85
ETP7	93596.42	91624.57
ETP9	124413.52	78413.22
ETP10	268200.75	172964.27
ETP11	182041.66	241171.98
ETP12	96130.34	47407.41

performs SBGP-HH for 6 out of 9 instances. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for the other 3 target problem instances, resulting in a negative effect on the objective value. This result was found to be statistically significant for ETP4, ETP7, ETP9, ETP10 and ETP12.

Table 13 shows the computational effort values for both approaches. The target generation where a solution is expected to be found has been set to 80. The computational effort needed by SBGP-HH-TL is less than SBGP-HH for 5 out of 9 instances. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for the target problem instances where a higher computational effort was found, resulting in more generations required for SBGP-HH-TL to find the best solution.

The values for the generality for SBGP-HH and SBGP-HH-TL are 14.938 and 14.316 respectively. These results suggest that SBGP-HH-TL demonstrates superior generalisation capabilities compared to SBGP-HH-TL when applied to the 9 target problem instances of the ITC 2007 dataset.

6.2.2 One-dimensional bin packing problem

Table 14 provides the number of instances solved to optimum for SBGP-HH and SBGP-HH-TL on the ODBPP datasets. SBGP-HH-TL solved more instances than SBGP-HH to optimality in 3 out of 5 datasets, including the Scholl Hard dataset.

Table 14 SBGP-HHvs SBGP-HH-TLperformance on ODBPP datasets. Best values are highlighted in bold

Datasets	SBGP-HH			SBGP-HH-TL		
	Opt	Opt.-1	Sum	Opt	Opt.-1	Sum
Scholl Easy (713)	626	79	705	621	88	709
Scholl Medium (475)	376	61	437	377	62	439
Scholl Hard (10)	7	3	10	8	2	10
Falkenauer_u (80)	30	39	69	31	40	71
Falkenauer_t (80)	0	55	55	0	50	50
Total (1358)	1039	237	1276	1037	242	1279

Table 15 SBGP-HH vs SBGP-HH-TL average computational effort on ODBPP datasets. Best values are highlighted in bold

	SBGP-HH	SBGP-HH-TL
Scholl Easy	4246.311	4797.355
Scholl Medium	23550.836	26567.951
Scholl Hard	24069.791	25362.396
Falkenauer_u	21315.83	22671.067
Falkenauer_t	18986.84	15392.831

SBGP-HH-TL solved the most instances of the Scholl Hard dataset to optimality compared to SBGP-HH of which each only solved 7 instances. Overall, SBGP-HH solved 2 instances to optimality more than SBGP-HH-TL. These results were found to be statistically significant.

Table 15 presents the average computational effort for each dataset. The generation by which a solution program is found is 50. From the table, the computational effort needed by SBGP-HH is less than SBGP-HH-TL on 4 datasets. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for these the target problem instances.

For the 1358 instances, the generality for SBGP-HH and SBGP-HH-TL is 1.203 and 1.206, respectively. This 0.003 difference suggests that there is not a major difference in generalisation between SBGP-HH and SBGP-HH-TL.

6.2.3 Capacitated Vehicle Routing Problem

Table 16 provides the minimum objective values, averages and standard deviations of 30 runs of SBGP-HH vs. SBGP-HH-TL on the CVRP Christofides dataset. SBGP-HH-TL outperforms SBGP-HH in all 10 instances. The source problem instances are not included in the table as the SBGP-HH-TL did not run on these instances. These results were found to be significant for all problem instances.

Table 17 shows the computational effort values for both approaches. From the table, the computational effort needed by SBGP-HH is less than SBGP-HH-TL on 7 instances. This observation indicates that SBGP-HH finds in most cases the best solution of a run in earlier generations than SBGP-HH-TL. It is hypothesised that the transferred knowledge provides genetic material to enable SBGP-HH-TL to find better solutions than SBGP-HH in later generations of a run.

For the 10 instances the generality SBGP-HH and SBGP-HH-TL is 3.827 and 3.579 respectively. These results suggest that SBGP-HH-TL demonstrates superior

Table 16 SBGP-HH vs SBGP-HH-TL performance on CVRP Christofides over 30 runs. Best values are highlighted in bold

Instance	SBGP-HH			SBGP-HH-TL		
	Best	AVE	STD	Best	AVE	STD
CVRP3	891.7708	925.2	17.1	866.2857	906.8	16.4
CVRP4	1164.0816	1208.1	18.5	1130.9377	1179.2	20.8
CVRP7	970.2701	1012.9	15.9	967.1423	982.2	11.8
CVRP8	944.3633	980.6	15.4	914.8784	949.9	16.4
CVRP9	1310.8663	1344.9	15.4	1297.4159	1320.8	11.3
CVRP10	1579.271	1612.2	26.2	1568.9758	1591.8	15
CVRP11	1109.4358	1156.5	27.3	1074.2877	1116.9	21.2
CVRP12	838.2984	852.8	6.4	837.2731	844.9	5.2
CVRP13	1599.899	1616.3	9.6	1584.0482	1604.3	8.7
CVRP14	894.0796	934.8	16	891.1785	912.6	11.7

Table 17 SBGP-HHvs SBGP-HH-TL Computational Effort for the Christofides dataset. Best values are highlighted in bold

Instance	SBGP-HH	SBGP-HH-TL
CVRP3	85408.06	73879.41
CVRP4	33666.67	95260.18
CVRP7	49179.51	46326.43
CVRP8	51815.12	71978.97
CVRP9	69840.98	155969.96
CVRP10	38986.28	79479.71
CVRP11	97991.19	232323.15
CVRP12	66109.29	63843.94
CVRP13	42259.26	136053.89
CVRP14	61638.32	102384.87

generalisation capabilities compared to SBGP-HH when applied to the Christofides dataset.

6.3 Performance Analysis of the Structure-Based Genetic Programming

This section provides an analysis of the performance of the hyper-heuristics in the context of the aims of this study. One of the aims of this study is to evaluate structure-based genetic programming compared to canonical genetic programming in generation constructive hyper-heuristics. The study has shown that generally SBGP-HH performed better than CGP-HH, however in some instances CGP-HH did perform better. We examine the reasons for this. In the cases where SBGP-HH performed better than CGP-HH the reason for this is that the exploration based on structure moved the search to a different area of the search space when a local optimum was reached. Figure 4 illustrates this. The figure plots the best fitness over a run for SBGP-HH and CGP-HH where SBGP-HH performs better than CGP-HH. From this image it is evident that SBGP-HH is able to escape local optima due to the global level search within the first twenty generations. However, this figure also shows that while an optimum was reached SBGP-HH then explored an area of worse fitness due to the global level search.

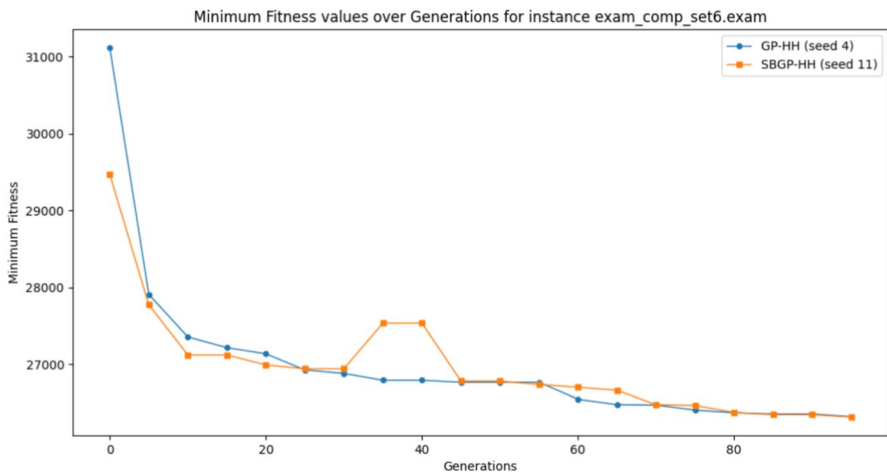


Fig. 4 SBGP-HH Outperforms CGP-HH

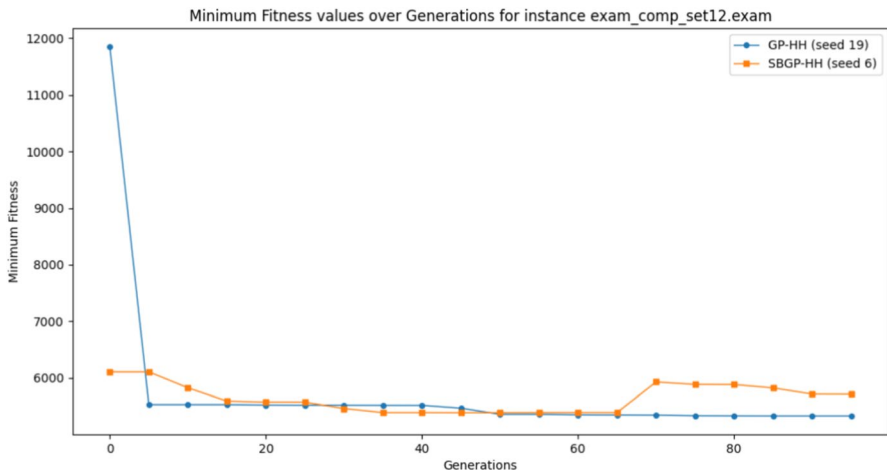


Fig. 5 CGP-HH Outperforms SBGP-HH

We have also examined instances where CGP-HH outperforms SBGP-HH. In these instances the global level search in SBGP-HH moves the search to an area that does not lead to a better optimum as indicated in Fig. 4. This is also illustrated in Fig. 5. Thus, SBGP-HH can possibly be improved by more intelligently determining the threshold when selecting global areas and improving the mechanism for selecting global areas. This will be investigated as part of future research.

The second aim of this study was to investigate the use of transfer learning in structure-based genetic programming. Hence, we examined instances when SBGP-HH-TL performed better than SBGP-HH and when SBGP-HH-TL performed worse to get a better understanding of the reasons for this.

In the cases that SBGP-HH-TL performed better than SBGP-HH was due to the search starting off with points in the space with better fitness due to the knowledge being transferred. However, beyond the initial population the global level search may direct the search to an area that does not lead to a better solution program. This has been the reason for SBGP-HH-TL not performing better than SBGP-HH. This can be seen in Fig. 6. As mentioned above future work will investigate a better mechanism for identifying global areas.

6.4 Comparison with Construction Heuristics and Hyper-Heuristics

This section provides comparisons between CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL with existing construction heuristics and constructive hyper-heuristics. The human-derived construction heuristics are defined in Appendix D and the construction hyper-heuristics in sections 2.4.2 and 2.5.2.

Table 18 compares the performance of human-derived construction heuristics and those generated by the hyper-heuristics for the ITC 2007 dataset. In the notation h/s , h represents the hard constraint cost and s the soft constraint cost. A hard constraint violation value of 0 implies a feasible solution. Values of the human-derived construction heuristics are from Pillay [3]. The structure-based hyper-heuristics have produced the best results with SBGP-HH-TL performing better than SBGP-HH.

Table 19 compares the performance of the proposed hyper-heuristics with other construction hyper-heuristics, both selection and generation described in 2.4.2. CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL collectively produced the best results for 7 out of 12 instances. Selection construction hyper-heuristics employed by Sabar et al. [27] and Rahman et al. [29] performed the best on 5 instances using. Despite CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL not performing optimisation, the heuristics generated achieved comparative results.

For the ODBPP datasets, Burke et al. [4] and Singh and Pillay [41] presented their results for a dataset as a *Ratio*. This *Ratio* has also been calculated for CGP-HH,

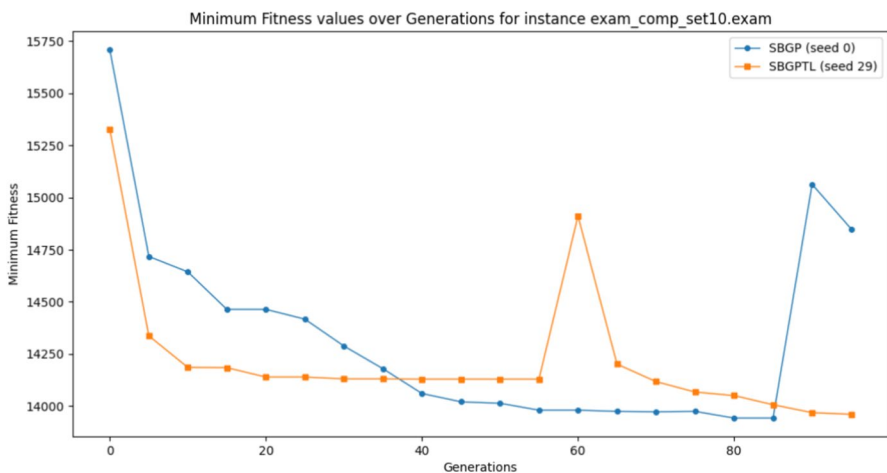


Fig. 6 SBGP-HH Outperforms SBGP-HH-TL

Table 18 Comparison of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL with construction heuristics'. A '-' indicates that this result is not available. A 'S' indicates that the algorithm was not applied to the problem instance as it was a source instance

Instance	ETP-LD	ETP-LWD	ETP-LE	ETP-LCD	ETP-SD	CGP-HH	SBGP-HH	CGP-HH-TL	SBGP-HH-TL
ETP1	0 / 12870	0 / 11519	0 / 12360	0 / 12609	0 / 12600	0 / 5198	0 / 5176	S	S
ETP2	0 / 7492	7 / 18343	1 / 5668	0 / 4112	0 / 4599	0 / 478	0 / 492	S	S
ETP3	8 / 18397	7 / 18343	9 / 22153	0 / 16568	8 / 18508	0 / 10309	0 / 10184	0 / 10184	0 / 10164
ETP4	2 / 26457	50 / 38241	49 / 31238	4 / 25418	2 / 30683	0 / 17088	0 / 19319	0 / 19319	0 / 17821
ETP5	9 / 7189	10 / 13927	9 / 13128	0 / 8548	9 / 6801	0 / 3084	0 / 3067	0 / 3067	0 / 3083
ETP6	4 / 33545	6 / 33800	11 / 36855	0 / 35800	8 / 32930	0 / 26320	0 / 26300	0 / 26300	0 / 26270
ETP7	4 / 19048	5 / 18241	5 / 19604	0 / 17312	1 / 23561	0 / 5206	0 / 5065	0 / 5065	0 / 5000
ETP8	1 / 20084	0 / 15702	0 / 16302	0 / 15715	0 / 15336	0 / 9289	0 / 9277	S	S
ETP9	-	-	-	-	-	0 / 1097	0 / 1074	0 / 1074	0 / 1071
ETP10	-	-	-	-	-	0 / 14003	0 / 13942	0 / 13942	0 / 13954
ETP11	-	-	-	-	-	0 / 30109	0 / 30019	0 / 30019	0 / 30057
ETP12	-	-	-	-	-	0 / 5330	0 / 5388	0 / 5388	0 / 5339

Table 19 CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL vs. constructive hyper-heuristics. A '-' indicates that this value is not available. A 'S' indicates that the algorithm was not applied to the instance as it was a source instance

Instance	[31]	[26]	[27]	[28]	[3]	[29]	CGP-HH	SBGP-HH	CGP-HH-TL	SBGP-HH-TL
ETP1	8006	8559	6234	5752	9403	5231	5198	5176	S	S
ETP2	3470	830	395	1693	1891	433	478	492	S	S
ETP3	18622	11576	13002	14586	13884	9265	10309	10184	10184	10164
ETP4	22559	21901	17940	21491	18760	17787	17088	19319	19319	17821
ETP5	4714	3969	3900	3844	5551	3083	3084	3067	3067	3083
ETP6	29155	28340	27000	28480	29695	26060	26320	26300	26300	26270
ETP7	10473	8167	6214	5182	10569	10712	5206	5065	5065	5000
ETP8	14317	12658	8552	13711	14024	12713	9289	9277	S	S
ETP9	-	-	-	-	-	1111	1097	1074	1074	1071
ETP10	-	-	-	-	-	14825	14003	13942	13942	13954
ETP11	-	-	-	-	-	28891	30109	30019	30019	30057
ETP12	-	-	-	-	-	6181	5330	5388	5388	5339

SBGP-HH, CGP-HH-TL and SBGP-HH-TL to allow for comparison. Equations 7, 8 and 9 are used to calculate *Ratio*.

$$InstanceAverage = \frac{\sum_{i=1}^u r_i}{u} \quad (7)$$

$$InstanceRatio = \frac{InstanceAverage}{z} \quad (8)$$

$$Ratio = \frac{\sum_{i=1}^m InstanceRatio}{m} \quad (9)$$

For each instance, u runs were performed with different seeds for each. It is noted that results obtained from Singh and Pillay [41] ran for ten runs, while CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL ran for 30 runs. *InstanceAverage* is the average value over the runs for an instance. For each instance, *InstanceRatio* is calculated as the ratio of *InstanceAverage* to the best result from literature. Finally, *Ratio* is the average *InstanceAverage* on each problem instance m in a dataset. A *Ratio* of 1.0 implies that optimal values were obtained for each instance in the dataset. Falkenauer_u and Scholl Hard datasets were used in previous studies [4, 41], and used for comparison with construction heuristics that were presented in Singh and Pillay [41]. Table 20 presents these comparisons. From this table, CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL produced better results than all construction heuristics and the ant colony optimisation algorithm. The constructive hyper-heuristic from Burke et al. [4] performed the best of these constructive hyper-heuristics on these ODBPP datasets.

Table 21 provides the comparison of the results between CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL and construction heuristics and selection and generation constructive hyper-heuristics from previous work described in section 2.5.2. SBGP-HH-TL was applied to the source problem instances as well to allow for comparison. From the table it is evident that CGP-HH, SBGP-HH, CGP-HH-TL and

Table 20 'Ratio' results of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL compared with ODBPP construction heuristics's from Burke et al. [4] and Singh and Pillay [41]. Best values are highlighted in bold

Algorithm	Falkenauer_u		Scholl Hard	
	Ratio	SD	Ratio	SD
ODBPP-FF	1.0586	0.0113	1.0606	0.0099
ODBPP-BF	1.0545	0.0108	1.0606	0.0099
ODBPP-NF	1.3117	0.0175	1.1566	0.0117
ODBPP-WF	1.1477	0.02	1.0606	0.0099
ODBPP-FFD	1.0129	0.0062	1.0606	0.0099
ODBPP-BFD	1.0129	0.0062	1.0606	0.0099
ODBPP-NFD	1.4012	0.0195	1.1566	0.0117
[4]	1	0.0003	1.0004	0.007
[41]	1.0253	0.0077	1.0624	0.0088
CGP-HH	1.0099	0.006	1.0177	0.0135
SBGP-HH	1.0098	0.006	1.0157	0.0114
CGP-HH-TL	1.0061	0.0057	1.0132	0.0098
SBGP-HH-TL	1.0096	0.006	1.0159	0.0118

Table 21 CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL compared with ODBPP construction heuristics and generation construction hyper-heuristics

Approach	Opt. (1370)
ODBPP-FFD	788
ODBPP-DJD [61]	716
ODBPP-DJT [62]	863
ODBPP-ADJD [63]	686
[14]	1126
[40]	1209
CGP-HH	1013
SBGP-HH	1017
CGP-HH-TL	1016
SBGP-HH-TL	1014

Table 22 Output of CGP-HH, SBGP-HH, CGP-HH-TL, SBGP-HH-TL and construction heuristics's for a subset of the Christofides dataset. The best value is highlighted in bold

Approach	Output
CVRP-CW	4.6
CVRP-MJ	11.45
CVRP-GM	12.35
CVRP-KI	9.59
CGP-HH	7.3
SBGP-HH	7.28
CGP-HH-TL	5.31
SBGP-HH-TL	4.7

SBGP-HH-TL solved more instances to optimality than the construction heuristics. The generation construction hyper-heuristics employed by Sim and Hart [14] and Sim et al. [40] produced the best results.

Table 22 compares the performance of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL and CVRP construction heuristics for a subset of the Christofides dataset. The values of the heuristics are from Garrido et al. [64]. The output was calculated as the average deviation percentage from the best-known value of the respective instance. From the table, CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL performed better than 3 of the 4 construction heuristics.

6.5 Comparison with State of the Art Approaches

This section compares the performance of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL to the state of the art approaches as described in sections 2.4.3, 2.5.3 and 2.6.2. These state of the art algorithms perform optimisation in the solution space whereas constructive hyper-heuristics provide an initial point in the solution space.

It is emphasised that this study does not intend to compete with the state of the art algorithms. Thus, the comparison is included for completeness and it is not expected that the performance of the derived heuristics will be comparable to state of the art approaches.

Table 23 shows CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL compared to the current state of the art for the ITC 2007 dataset.

Tables 24 and 25 present a comparison of the state of the art approaches for the ODBPP datasets. In order to compare with these approaches that ran on all 1370

Table 23 CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL results compared to state-of-the-art algorithms applied to the ITC2007 dataset. Cells marked with 'S' indicate that the algorithm did not run on the problem instance because it is where knowledge was transferred from

Instance	[30]	[32]	[33]	[34]	[35]	CGP-HH	SBGP-HH	CGP-HH-TL	SBGP-HH-TL
ETP1	4370	4128	3647	3925	3085	5198	5176	S	S
ETP2	400	380	385	376	380	478	492	S	S
ETP3	10049	7769	7487	8245	7147	10309	10184	10184	10164
ETP4	18141	13103	11779	12493	10448	17088	19319	19319	17821
ETP5	2988	2533	2447	2659	2341	3084	3067	3067	3083
ETP6	26585	25330	25210	25205	25120	26320	26300	26300	26270
ETP7	4213	3537	3563	3901	3225	5206	5065	5065	5000
ETP8	7742	7087	6614	6756	7077	9289	9277	S	S
ETP9	1030	913	924	910	898	1097	1074	1074	1071
ETP10	16682	13053	12931	12439	13264	14003	13942	13942	13954
ETP11	34129	24369	23784	24809	22343	30109	30019	30019	30057
ETP12	5535	5095	5097	5095	5095	5330	5388	5388	5339

Table 24 Comparison of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL with state-of-the-art algorithms applied to the ODBPP Scholl datasets. Values indicate the number of instances that were solved to optimality. Best values are highlighted in bold

Approach	Scholl Easy (720)	Scholl Medium (480)	Scholl Hard (10)
[42]	720	480	10
[43]	720	480	10
[44]	667	412	8
[45]	720	480	10
[46]	720	480	10
CGP-HH	626	380	7
SBGP-HH	630	380	7
CGP-HH-TL	634	375	7
SBGP-HH-TL	625	381	8

Table 25 Comparison of CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL with state-of-the-art algorithms applied to the ODBPP Falkenauer datasets. Values indicate the number of instances that were solved to optimality. Best values are highlighted in bold

Approach	Falkenauer_u (80)	Falkenauer_t (80)
[43]	79	80
[47]	78	-
CGP-HH	29	0
SBGP-HH	30	0
CGP-HH-TL	30	0
SBGP-HH-TL	31	0

Scholl instances, CGP-HH-TL and SBGP-HH-TL were applied to the source problem instances as well, and the results obtained are included here.

Table 26 shows a comparison with CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL and the state-of-the-art approaches applied to the CVRP Christofides dataset.

As expected the hyper-heuristics have not outperformed these approaches, but given that the hyper-heuristics only find an initial point they have performed well in comparison.

6.6 Discussion

From the results presented in the previous section it is evident that SBGP-HH-TL outperforms SBGP-HH, CGP-HH-TL and CGP-HH producing the best results for a majority of the problem instances for all three problem domains. In terms of computational effort, the study has revealed that whether SBGP-HH-TL reduces the computational effort or not is problem dependent. The differences in generality attained by CGP-HH, SBGP-HH and SBGP-HH-TL are small. CGP-HH generalises better than SBGP-HH and SBGP-HH-TL. This is expected as structure-based genetic programming performs global and local level searches to control exploration and exploitation for the problem at hand.

We have also examined the sensitivity of SBGP-HH-TL to the parameters for structure-based genetic programming. The analysis has shown that the changes in parameters do not have a drastic impact on the performance of the hyper-heuristic. An example of this is illustrated in Table 27 for the CVRP3. The “Result” refers to the objective value. As can be seen for while a different set of parameters may improve the results, this does not result in drastic performance changes. As with most meta-heuristics SBGP-HH-TL requires parameter tuning to produce the best results.

The global areas moved to as part of SBGP-HH were examined with the aim of identifying any patterns or recurring global areas producing good results. The analysis revealed that there were no such patterns. Based on how the global areas were selected, the best option varied per instance and in response to random noise due to the stochastic nature of structure-based genetic programming. As discussed in section 6.3 the mechanism used to select global areas needs to be improved and this will be addressed as part of future work.

Table 26 Comparison between CGP-HH, SBGP-HH, CGP-HH-TL and SBGP-HH-TL state-of-the-art algorithms applied to the CVRP Christofides dataset. Cells marked with ‘S’ indicate that the algorithm did not run on the problem instance because it is where knowledge was transferred from

Instance	Best known	Reference	CGP-HH	SBGP-HH	CGP-HH-TL	SBGP-HH-TL
CVRP1	524.61	[48, 51, 53]	555.4733	550.2551	S	S
CVRP2	835.26	[48, 51]	921.2329	908.6607	S	S
CVRP3	826.14	[48, 51]	906.1137	891.7708	883.8416	866.2857
CVRP4	1028.42	[48]	1149.7097	1164.0816	1132.04	1130.9377
CVRP5	1291.29	[50]	1453.6618	1449.0727	S	S
CVRP6	555.43	[48, 51, 53]	583.7999	571.3939	S	S
CVRP7	909.68	[48]	978.5827	970.2701	969.8392	967.1423
CVRP8	865.94	[48]	952.2692	944.3633	938.7198	914.8784
CVRP9	1162.55	[48]	1295.6049	1310.8663	1286.4329	1297.4159
CVRP10	1395.85	[50]	1570.4997	1579.271	1563.4395	1568.9758
CVRP11	1042.12	[48, 51]	1082.6853	1109.4358	1080.8988	1074.2877
CVRP12	819.56	[48, 51, 53]	835.5178	838.2984	836.1166	837.2731
CVRP13	1541.14	[48]	1598.1916	1599.899	1579.7052	1584.0482
CVRP14	866.37	[48, 53]	907.9564	894.0796	885.9904	891.1785

Table 27 SBGP-HH-TL Parameter Analysis

Parameter	Value	Result	Value	Result	Value	Result	Value	Result
S_g	5	869.1377	10	866.2857	20	883.3644	30	867.8271
D_g	3	870.2856	4	866.2857	5	871.8386	6	869.0787
T_g	3	870.5842	6	866.2857	12	882.308	18	878.777
W_g	5	878.3833	10	866.2857	15	878.3606	20	881.7929
DI_g	4	875.3075	5	882.2096	6	866.2857	7	873.9063

7 Conclusion and Future Work

The main aim of the research presented in this paper was to investigate the use of structured-based genetic programming and structured-based genetic programming with transfer learning in generation constructive hyper-heuristics for combinatorial optimisation. The study has shown the effectiveness of structure-based genetic programming generation constructive hyper-heuristics for three combinatorial optimisation problems, namely, examination timetabling, one-dimensional bin packing and the capacitated vehicle routing problem.

The study has also highlighted possible areas of improvement. One of the challenges identified in this study is an appropriate mechanism to identify global areas for global level search. The study has also shown that structure-based genetic programming does not guarantee an improvement in computational effort. Future research will also investigate methods for improving computational effort. Further investigation into what is the best knowledge to transfer and when to transfer will also be conducted. Determining the appropriate source domains to use for the a target domain to ensure positive transfer will also be investigated.

Appendix A - Canonical Genetic Programming (CGP-HH) vs. Canonical Genetic Programming with Transfer Learning (CGP-HH-TL)

Examination Timetabling Problem

Table 28 provides the minimum objective values, averages and standard deviations of 30 runs of CGP-HH vs. CGP-HH-TL on the ETP ITC 2007 dataset. It is emphasised here that the source problem instances are not included in the table for the CGP-HH-TL did generate heuristics for these instances. From the table, it is evident that CGP-HH-TL outperforms CGP-HH in 6 out of 9 instances. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for the other 3 target problem instances, resulting in a negative effect on the objective value Table 29

Table 30 shows the computational effort values for both approaches. TThe generation by which a solution program is found is 80. From the table, the computational effort needed by CGP-HH is less than CGP-HH-TL on 7 instances. Among the instances where CGP-HH had a lower computational effort than CGP-HH-TL, CGP-HH also performed better than CGP-HH-TL, as shown in Table 28. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for

Table 28 CGP-HH vs CGP-HH-TL Performance on ETP ITC 2007 over 30 runs. Best values are highlighted in bold

Instance	CGP-HH			CGP-HH-TL		
	Best	AVE	STD	Best	AVE	STD
ETP3	10309	10541.7	177.3	9940	10572.1	255.4
ETP4	17088	32634.8	6641.9	18542	23045.5	5196.8
ETP5	3084	3292.8	98.6	2998	3230.6	102.1
ETP6	26320	26718.2	176.7	26305	26639.2	193.9
ETP7	5206	5242.2	102.9	5030	5167.4	67.5
ETP9	1097	1131.3	19.4	1060	1125.8	31
ETP10	14003	14202.2	87	14017	14220.9	170.2
ETP11	30109	31339.7	668.5	29830	31435.4	884.7
ETP12	5330	5410.3	54.5	5348	5473.9	116.6

Table 29 CGP-HHvs CGP-HH-TLp-values for the ETP ITC 2007 dataset

Instance	P value
ETP3	0.159
ETP4	1
ETP5	0.989
ETP6	0.944
ETP7	0.999
ETP9	0.88
ETP10	0.661
ETP11	0.438
ETP12	0.023

Table 30 CGP-HH vs CGP-HH-TL Computation Effort for the ETP ITC 2007 dataset. Best values are highlighted in bold

Instance	CGP-HH	CGP-HH-TL
ETP3	312550.96	298802.38
ETP4	121319.24	280470.95
ETP5	231075.77	231699.46
ETP6	128882.75	189952.02
ETP7	184920.66	293597.76
ETP9	78868.05	88054
ETP10	200423	282850.37
ETP11	405274.26	225462.6
ETP12	58446.22	58651.84

the target problem instances where a higher computational effort was found, resulting in more generations required for CGP-HH-TL to find the best solution.

The values for the generality performance metric for CGP-HH and CGP-HH-TL are 13.215 and 13.896 respectively. These results suggest that CGP-HH demonstrates superior generalisation capabilities compared to CGP-HH-TL when applied to the 9 target problem instances of the ITC 2007 dataset.

One-Dimensional Bin Packing Problem

Table 31 provides the number of instances solved to optimum by CGP-HH and CGP-HH-TL on the ODBPP datasets. From the table, CGP-HH-TL solved more

Table 31 CGP-HH vs CGP-HH-TL performance on ODBPP datasets. Best values are highlighted in bold

datasets	CGP-HH			CGP-HH-TL		
	Opt	Opt.-1	Sum	Opt	Opt.-1	Sum
Scholl Easy (713)	622	87	709	630	78	708
Scholl Medium (475)	376	60	436	371	67	438
Scholl Hard (10)	7	3	10	7	3	10
Falkenauer_u (80)	29	44	73	30	44	74
Falkenauer_t (80)	0	53	53	0	52	52
Total (1358)	1034	247	1281	1038	244	1282

Table 32 CGP-HH vs CGP-HH-TL average computational effort on ODBPP datasets. Best values are highlighted in bold

	CGP-HH	CGP-HH-TL
Scholl Easy	2739.82	2120.718
Scholl Medium	12568.339	7311.419
Scholl Hard	13508.437	11751.594
Falkenauer_u	13939.966	6702.621
Falkenauer_t	11023.883	3571.321

instances to optimality than CGP-HH in all datasets. CGP-HH-TL only solved 4 more instances than CGP-HH to optimality.

As with ETP, statistical tests were performed for each problem instance of ODBPP. From the tests, CGP-HH produced significantly higher values for 18% (249 out of 1358) and 45% (616 out of 1358) of instances for the number of bins and packing efficiency respectively.

Table 32 presents the average computational effort for each dataset. The target generation where a solution is expected to be found has been set to 50. From the table, the computational effort needed by CGP-HH-TL is less than CGP-HH on each dataset. From the table, less computational effort was required for each dataset by CGP-HH-TL than CGP-HH. It is hypothesised that that the knowledge transferred to CGP-HH-TL enabled it to converge earlier.

For the 1358 instances, values for the generality performance metric are for CGP-HH and CGP-HH-TL as 1.221 and 1.334 respectively. These results suggest that CGP-HH demonstrates superior generalisation capabilities compared to CGP-HH-TL when applied to the ODBPP datasets. It is hypothesised that the knowledge transferred to the target problem instances was not appropriate for the target problem instances, resulting in more generations required for CGP-HH-TL to find a best solution.

Capacitated Vehicle Routing Problem

Table 33 provides the minimum objective values, averages and standard deviations of 30 runs of CGP-HH vs. CGP-HH-TL on the CVRP Christofides dataset. From the table, CGP-HH-TL outperforms CGP-HH in 9 out of 10 instances. It is emphasised here that the source problem instances are not included in the table for the CGP-HH-TL did not evolve heuristics for these instances.

A statistical test was performed for performance comparison of CGP-HH and CGP-HH-TL for each instance of the Christofides dataset. Table 34 presents the p-values for each problem instance. From the table, the null hypothesis is rejected for

Table 33 CGP-HH vs CGP-HH-TL performance on CVRP Christofides over 30 runs. Best values are highlighted in bold

Instance	CGP-HH			CGP-HH-TL		
	Best	AVE	STD	Best	AVE	STD
CVRP3	906.1137	930.8	13.4	883.8416	901.6	10.9
CVRP4	1149.7097	1210.9	24.4	1132.04	1153.1	19.4
CVRP7	978.5827	1018.9	11.7	969.8392	988.6	15.7
CVRP8	952.2692	985.4	25.2	938.7198	954.8	8.4
CVRP9	1295.6049	1344	24.5	1286.4329	1312.9	13.3
CVRP10	1570.4997	1614.1	30.9	1563.4395	1591.3	15.9
CVRP11	1082.6853	1133.3	25.1	1080.8988	1116.7	22.7
CVRP12	835.5178	848.9	7	836.1166	844.8	4.2
CVRP13	1598.1916	1615.4	11.8	1579.7052	1596.2	7.9
CVRP14	907.9564	936.9	17.7	885.9904	901.9	9.5

Table 34 CGP-HH vs CGP-HH-TL p-values for the Christofides dataset

Instance	P value
CVRP3	0.999999992
CVRP4	0.999999982
CVRP7	0.999999913
CVRP8	0.999999913
CVRP9	0.999999486
CVRP10	0.99928
CVRP11	0.99484
CVRP12	0.99119
CVRP13	0.9999999784
CVRP14	0.999999999

all 10 instances, where CGP-HH has a significantly higher mean objective value than CGP-HH-TL. Despite the fact that CGP-HH-TL performed worse than one instance, as shown in Table 33, CGP-HH still produced a significantly higher mean objective value than CGP-HH-TL.

Table 35 shows the computational effort for both approaches. From the table, the computational effort needed by CGP-HH is less than CGP-HH-TL for all 10 instances. This observation indicates that CGP-HH finds in most cases the best solution of a run in earlier generations than CGP-HH-TL. It is hypothesised that the transferred knowledge provides genetic material to enable CGP-HH-TL to find better solutions than CGP-HH in later generations of a run.

For the 10 instances, the values for the generality performance metric for CGP-HH and CGP-HH-TL are 3.604 and 3.506, respectively. This suggests that CGP-HH-TL demonstrates superior generalisation capabilities compared to CGP-HH when applied to the Christofides dataset.

Table 35 CGP-HHvs CGP-HH-TL Computational Effort for the Christofides dataset. Best values are highlighted in bold

Instance	CGP-HH	CGP-HH-TL
CVRP3	14650	40454.86
CVRP4	20740.62	94076.26
CVRP7	13773.3	75067.18
CVRP8	20823.36	27546.59
CVRP9	46616.21	151060.5
CVRP10	38283.03	54902.21
CVRP11	111686.29	192323.22
CVRP12	63247.83	77793.04
CVRP13	73401.27	174286.05
CVRP14	49710.81	319425.24

Table 36 CGP-HH-TL vs SBGP-HH-TL performance on ETP ITC 2007 over 30 runs. Best values are highlighted in bold

Instance	GPTL			SBGPTL		
	Best	AVE	STD	Best	AVE	STD
ETP3	9940	10572.1	255.4	10164	10585.8	208
ETP4	18542	23045.5	5196.8	17821	25247.4	6605.9
ETP5	2998	3230.6	102.1	3083	3247.1	81.1
ETP6	26305	26639.2	193.9	26270	26623.4	184.5
ETP7	5030	5167.4	67.5	5000	5224.6	94.5
ETP9	1060	1125.8	31	1071	1110.7	21.1
ETP10	14017	14220.9	170.2	13954	14184.9	104.2
ETP11	29830	31435.4	884.7	30057	31680.5	1022.4
ETP12	5348	5473.9	116.6	5339	5424.7	72.8

Appendix B - Canonical Genetic Programming with Transfer Learning (CGP-HH-TL) vs. Structure-Based Genetic Programming with Transfer Learning (SBGP-HH-TLGPTL)

Examination Timetabling Problem

Table 36 provides the minimum objective values, averages and standard deviations of 30 runs of CGP-HH-TL vs. SBGP-HH-TL on the ETP ITC 2007 dataset. It is emphasised here that the source problem instances are not included in the table for SBGP-HH-TL and CGP-HH-TL did not generate heuristics for these instances. From the table, it is evident that SBGP-HH-TL outperforms CGP-HH-TL in 5 out of 9 instances. These results were found to be statistically significant for two problem instances.

Table 38 shows the computational effort values for both approaches. The generation by when a solution program is found is 80. From the table, the computational effort needed by SBGP-HH-TL is less than CGP-HH-TL on 7 instances, with one draw. Of the four instances where SBGP-HH-TL outperforms CGP-HH-TL, as shown in Table 37, SBGP-HH-TL also shows better computational effort than CGP-HH-TL.

The generality performance for CGP-HH-TL and SBGP-HH-TL are 13.896 and 14.316 respectively. These results suggest that CGP-HH-TL demonstrates superior generalisation capabilities compared to SBGP-HH-TL when applied to the 9 target

Table 37 CGP-HH-TL vs SBGP-HH-TL p-values for the ETP ITC 2007 dataset

Instance	P value
ETP3	0.5588
ETP4	0.1683
ETP5	0.2822
ETP6	0.5543
ETP7	0.0054
ETP9	0.9876
ETP10	0.5762
ETP11	0.1777
ETP12	0.9623

Table 38 CGP-HH-TL vs SBGP-HH-TL Computational Effort for the ETP ITC 2007 dataset. Best values are highlighted in bold

Instance	CGP-HH-TL	SBGP-HH-TL
ETP3	298802.38	106006.55
ETP4	280470.95	280470.95
ETP5	231699.46	222181.57
ETP6	189952.02	103692.85
ETP7	293597.76	91624.57
ETP9	88054	78413.22
ETP10	282850.37	172964.27
ETP11	225462.6	241171.98
ETP12	58651.84	47407.41

Table 39 CGP-HH-TL vs SBGP-HH-TL performance on ODBPP datasets. Best values are highlighted in bold

datasets	CGP-HH-TL			SBGP-HH-TL		
	Opt	Opt.-1	Sum	Opt	Opt.-1	Sum
Scholl Easy (713)	630	78	708	621	88	709
Scholl Medium (475)	371	67	438	377	62	439
Scholl Hard (10)	7	3	10	8	2	10
Falkenauer_u (80)	30	44	74	31	40	71
Falkenauer_t (80)	0	52	52	0	50	50
Total (1358)	1038	244	1282	1037	242	1279

problem instances of the ITC 2007 dataset. These results were found to be statistically significant.

One-Dimensional Bin Packing Problem

Table 39 provides the number of instances solved to optimum of CGP-HH-TL vs. SBGP-HH-TL on the ODBPP datasets. From the table, SBGP-HH-TL solved the more instances to optimality than CGP-HH-TL in 3 out of 5 datasets, including the Scholl Hard dataset. SBGP-HH-TL solved the more instances of the Scholl Hard dataset to optimality than CGP-HH, SBGP-HH and CGP-HH-TL, of which each only solved 7 instances. Overall, CGP-HH-TL solved 1 instance more than SBGP-HH-TL, of which CGP-HH-TL solved 9 instances more to optimality than SBGP-HH-TL on the Scholl Easy dataset.

Table 15 presents the average computational effort for each dataset. The target generation where a solution is expected to be found has been set to 50. From the

Table 40 CGP-HH-TL vs SBGP-HH-TL average computational effort on ODBPP datasets. Best values are highlighted in bold

	CGP-HH-TL	SBGP-HH-TL
Scholl Easy	2120.718	4797.355
Scholl Medium	7311.419	26567.951
Scholl Hard	11751.594	25362.396
Falkenauer_u	6702.621	22671.067
Falkenauer_t	3571.321	15392.831

Table 41 CGP-HH-TL vs SBGP-HH-TL performance on CVRP Christofides over 30 runs. Best values are highlighted in bold

Instance	CGP-HH-TL			SBGP-HH-TL		
	Best	AVE	STD	Best	AVE	STD
CVRP3	883.8416	901.6	10.9	866.2857	906.8	16.4
CVRP4	1132.04	1153.1	19.4	1130.9377	1179.2	20.8
CVRP7	969.8392	988.6	15.7	967.1423	982.2	11.8
CVRP8	938.7198	954.8	8.4	914.8784	949.9	16.4
CVRP9	1286.4329	1312.9	13.3	1297.4159	1320.8	11.3
CVRP10	1563.4395	1591.3	15.9	1568.9758	1591.8	15
CVRP11	1080.8988	1116.7	22.7	1074.2877	1116.9	21.2
CVRP12	836.1166	844.8	4.2	837.2731	844.9	5.2
CVRP13	1579.7052	1596.2	7.9	1584.0482	1604.3	8.7
CVRP14	885.9904	901.9	9.5	891.1785	912.6	11.7

table, the computational effort needed by CGP-HH-TL is less than SBGP-HH-TL on all datasets. It is hypothesised that these results are due to the fact that the way SBGP-HH-TL uses transferred knowledge as compared to CGP-HH-TL causes it to use more generations to find the best solution Table 40.

For the 1358 instances generality for CGP-HH-TL and SBGP-HH-TL as 1.334 and 1.206 respectively. This indicates that SBGP-HH-TL demonstrates superior generalisation capabilities compared to CGP-HH-TL when applied to the ODBPP datasets.

Capacitated Vehicle Routing Problem

Table 41 provides the minimum objective values, averages and standard deviations of 30 runs of CGP-HH-TL vs. SBGP-HH-TL on the CVRP Christofides dataset. From the table, SBGP-HH-TL outperforms CGP-HH-TL in 5 out of 10 instances. It is emphasised here that the source problem instances are not included in the table for the SBGP-HH-TL and CGP-HH-TL did not run on these instances.

A statistical test was performed between CGP-HH-TL and SBGP-HH-TL for each instance of the Christofides dataset. Table 42 presented the p-values for each problem instance. From the table, the null hypothesis is rejected for 2 out of 10 instances, where CGP-HH-TL has a significantly higher mean objective value than SBGP-HH-TL. Although SBGP-HH-TL outperforms CGP-HH-TL on 5 out of 10 instances, as shown in Table 41, Table 42 shows that SBGP-HH-TL produced significantly better results for only 2 of those 5 instances.

Table 43 shows the computational effort values for both approaches. The target generation where a solution is expected to be found has been set to 80. From the table, the computational effort needed by SBGP-HH-TL is less than CGP-HH-TL on 4 out of 10 instances. This observation indicates that CGP-HH-TL finds in most cases

Table 42 CGP-HH-TL vs SBGP-HH-TL p-values for the Christofides dataset. Values higher than the critical value are highlighted in bold

Instance	P value
CVRP3	0.083
CVRP4	0
CVRP7	0.963
CVRP8	0.97
CVRP9	0.018
CVRP10	0.582
CVRP11	0.479
CVRP12	0.715
CVRP13	0
CVRP14	0.001

Table 43 CGP-HH-TL vs SBGP-HH-TL Computational Effort for the Christofides dataset. Best values are highlighted in bold

Instance	CGP-HH-TL	SBGP-HH-TL
CVRP3	40454.86	73879.41
CVRP4	94076.26	95260.18
CVRP7	75067.18	46326.43
CVRP8	27546.59	71978.97
CVRP9	151060.5	155969.96
CVRP10	54902.21	79479.71
CVRP11	192323.22	232323.15
CVRP12	77793.04	63843.94
CVRP13	174286.05	136053.89
CVRP14	319425.24	102384.87

the best solution of a run in earlier generations than SBGP-HH-TL. Of the 5 instances where SBGP-HH-TL outperformed CGP-HH-TL, as shown in Table 41, SBGP-HH-TL only produced a better computational effort for 1 of the 5 instances.

For the 10 instances, the values for the generality performance metric for CGP-HH-TL and SBGP-HH-TL are 3.506 and 3.579, respectively. These results suggest that CGP-HH-TL demonstrates superior generalisation capabilities compared to SBGP-HH when applied to the Christofides dataset.

Appendix C - Statistical Test Results

This appendix lists the p-values for the statistical tests conducted for the comparison of hyper-heuristic performance.

Performance Comparison of CGP-HH and SBGP-HH

Table 44 lists the p-values for the result that SBGP-HH performs better than CGP-HH for the ETP problem instances.

Table 45 lists the p-values for the result that SBGP-HH performs better than CGP-HH for the CVRP problem instances.

Table 44 CGP-HHvs SBGP-HHp-values for the ETP ITC 2007 dataset

Instance	P value
ETP1	0.8242
ETP2	0.0099
ETP3	0.1556
ETP4	0.9423
ETP5	0.9047
ETP6	0.9034
ETP7	0.1956
ETP8	0.5588
ETP9	0.9295
ETP10	0.1469
ETP11	0.0834
ETP12	0.0144

Table 45 CGP-HH vs SBGP-HH p-values for the Christofides dataset

Instance	P value
CVRP1	0.9473
CVRP2	0.9555
CVRP3	0.8615
CVRP4	0.9043
CVRP5	0.6219
CVRP6	0.9334
CVRP7	0.961
CVRP8	0.768
CVRP9	0.326
CVRP10	0.5266
CVRP11	0.0009
CVRP12	0.0128
CVRP13	0.3586
CVRP14	0.5791

Performance Comparison of SBGP-HH and SBGP-HH-TL

Table 46 lists the p-values for the result that SBGP-HH performs better than SBGP-HH-TL for the ETP problem instances.

Table 46 lists the p-values for the result that SBGP-HH performs better than SBGP-HH-TL for the CVRP problem instances Table 47.

Appendix D - Construction Heuristics

This appendix defines the human-derived heuristics construction heuristics that the performance of the heuristics created by the hyper-heuristics are compared to in section 6.4.

Examination Timetabling Heuristics [26, 65]:

- Saturation degree (ETP-SD) - exams are ordered in ascending order according

Table 46 SBGP-HHvs SBGP-HH-TLp-values for the ETP ITC 2007 dataset

Instance	P value
ETP3	0.6106
ETP4	0.9811
ETP5	0.5841
ETP6	0.8225
ETP7	0.9528
ETP9	0.9065
ETP10	0.9294
ETP11	0.5088
ETP12	0.9902

Table 47 SBGP-HHvs SBGP-HH-TLp-values for the Christofides dataset. Values higher than the critical value are highlighted in bold

Instance	P-Values
CVRP3	0.9999
CVRP4	0.99999846
CVRP7	0.999999876
CVRP8	0.999999859
CVRP9	0.999999881
CVRP10	0.999818
CVRP11	0.99999959
CVRP12	0.99999788
CVRP13	0.9999893
CVRP14	0.99999674

to the number of the remaining feasible slots during the timetable construction.

- Largest colour degree (ETP-LCD) - exams are ordered in descending order according to the number of conflicts with those already assigned.
- Largest enrolment first (ETP-LE) - exams are ordered in descending order according to the number of students enrolled.
- Largest degree first (ETP-LD) - exams are ordered in descending order according to the number of conflicts.
- Largest weighted degree (ETP-LWD) - exams are ordered in descending order according to the number of students involved in the conflicts. Construction heuristics for the one-dimensional bin packing problem [37, 38] (note that the items are ordered according to how they are received in a problem instance):
- Best Fit (ODBPP-BF) - Packs the next item into the bin that will cause the minimum remaining space over all bins.
- First Fit (ODBPP-FF) - In the sequence in which bins are opened, the next item is packed into the first bin it is able to fit in.
- Worst Fit (ODBPP-WF) - Packs the next item into the bin which will cause the maximum remaining space over all bins.
- Next Fit (ODBPP-NF) - In the sequence in which bins are opened and starting from the last bin that received an item, the next item is packed into the next bin that it can fit in.
- First Fit Decreasing (ODBPP-FFD) - In the sequence in which bins are opened, packs the largest remaining item into the first bin it can fit in.

- Next Fit Decreasing (ODBPP-NFD) - In the sequence in which bins are opened, packs the largest remaining item into the next bin it can fit in, starting from the last bin where an item was placed in. If the item does not fit in any of the remaining bins, then a new bin is opened and the item is placed there.
- Best Fit Decreasing (ODBPP-BFD) - In the sequence in which bins are opened, packs the largest item into the bin it can fit in and cause the least amount of wasted space.
- Djang and Fitch (ODBPP-DJD) [61] - Packs items into a bin until it is at most two-thirds full. Afterwards, a combination of three items is searched for to be packed into the bin to minimise remaining space.
- ODBPP-DJD with more Tuples (ODBPP-DJT) [62] - Same as ODBPP-DJD, but requires a combination of five items to be packed instead of three.
- Adaptive ODBPP-DJD (ODBPP-ADJD) [63] - repeatedly packs items into a bin until the remaining space in this bin is at most three times the average size of items that are not packed yet. The following construction heuristics have been widely used in the travelling salesman problem and its variations, like the CVRP:
- Kilby et al. [66] (CVRP-KI) - Inserts a customer into the route that has the lowest cost.
- Clark and Wright [67] (CVRP-CW) - Merges two routes into one if the distance to and from the depot is reduced as a result.
- Mole and Jameson [68] (CVRP-MJ) - Sequentially inserts a customer into the route that will result in the lowest cost increase.
- Gillet and Miller [69] (CVRP-GM) - Clusters routes by starting at an angle with the depot and north, then scans over all customers to find clusters. Clusters are limited by vehicle capacities. The shortest distance between the customers in each cluster is then searched for.

Author Contributions The first author conducted the research towards his Master's degree in Computer Science. The second other supervised the research and the writing of the paper.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

References

1. M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A.M. Karimi-Mamaghan, E.-G. Talbi, Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. *Eur. J. Oper. Res.* **296**(2), 393–422 (2022). <https://doi.org/10.1016/j.ejor.2021.04.032>
2. N. Pillay, R. Qu, *Hyper-Heuristics: Theory and Applications*, 1st edn. (Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2018)
3. N. Pillay, E. Özcan, Automated generation of constructive ordering heuristics for educational timetabling. *Ann. Oper. Res.* **275**, 181–208 (2019)

4. E.K. Burke, M.R. Hyde, G. Kendall, J. Woodward, Automating the packing heuristic design process with genetic programming. *Evol. Comput.* **20**(1), 63–89 (2012)
5. J.R. Koza, Genetic programming: on the programming of computers by means of natural selection (complex adaptive systems). *Bradford Book* **1**, 18 (1993)
6. D. Scheepers, N. Pillay, A study of transfer learning in a generation constructive hyper-heuristic for one dimensional bin packing, in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)* (IEEE, 2021), p. 1–7
7. M.A. Ardeh, Y. Mei, M. Zhang, Transfer learning in genetic programming hyper-heuristic for solving uncertain capacitated arc routing problem, in *2019 IEEE Congress on Evolutionary Computation (CEC)*, (2019), pp. 49–56 <https://doi.org/10.1109/CEC.2019.8789920>
8. Mei, Y., Ardeh, M.A., Zhang, M.: Knowledge transfer in genetic programming hyper-heuristics. *Automated Design of Machine Learning and Search Algorithms*, (2021), p. 149–169
9. N. Pillay, An investigation into the use of genetic programming for the induction of novice procedural programming solution algorithms in intelligent programming tutors. PhD thesis, School of Geological and Computer Sciences. University of KwaZulu-Natal (2004)
10. R. Kapoor, N. Pillay, Iterative structure-based genetic programming for neural architecture design. In: *Genetic and Evolutionary Computation—GECCO 23: companion lisabon, Portugal, July 15–19, 2023 Proceedings*, pp. 595–598. Springer (2023)
11. R. Kapoor, N. Pillay, A genetic programming approach to the automated design of cnn models for image classification and video shorts creation. *Genet. Prog. Evolvable Mach.* **25**(1), 10 (2024)
12. T.T. Huong Dinh, T.H. Chu, Q.U. Nguyen, Transfer learning in genetic programming, in *2015 IEEE Congress on Evolutionary Computation (CEC)*, (2015), pp. 1145–1151 <https://doi.org/10.1109/CEC.2015.7257018>
13. N. Pillay, Advances in evolutionary hyper-heuristics. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion. GECCO '25 Companion*, Association for Computing Machinery, New York, NY, USA, pp. 1579–1591. (2025). <https://doi.org/10.1145/3712255.3716520>
14. K. Sim, E. Hart, Generating single and multiple cooperative heuristics for the one dimensional bin packing problem using a single node genetic programming island model. In: *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation*, pp. 1549–1556 (2013)
15. D. Beckedahl, N. Pillay, A study of bi-space search for solving the one-dimensional bin packing problem. In: *International Conference on Artificial Intelligence and Soft Computing*, pp. 277–289 (2020). Springer
16. E.K. Burke, M.R. Hyde, G. Kendall, Evolving bin packing heuristics with genetic programming. In: *International Conference on Parallel Problem Solving from Nature*, pp. 860–869. Springer (2006)
17. M. Hyde, A genetic programming hyper-heuristic approach to automated packing PhD thesis, School of Computer Science, University of Nottingham (2010)
18. R. Ribani, M. Marengoni, A survey of transfer learning for convolutional neural networks, pp. 47–57 (2019). <https://doi.org/10.1109/SIBGRAP-T.2019.00010>
19. M. Iqbal, B. Xue, H. Al-Sahaf, M. Zhang, Cross-domain reuse of extracted knowledge in genetic programming for image classification. *IEEE Trans. Evol. Comput.* **21**(4), 569–587 (2017). <https://doi.org/10.1109/TEVC.2017.2657556>
20. B. Al-Helali, Q. Chen, B. Xue, M. Zhang, Multitree genetic programming with feature-based transfer learning for symbolic regression on incomplete data. *IEEE Trans. Cybern.* **54**(7), 4014–4027 (2024). <https://doi.org/10.1109/TCYB.2023.3270319>
21. B. Al-Helali, Q. Chen, B. Xue, M. Zhang, Genetic programming for feature selection based on feature removal impact in high-dimensional symbolic regression. *IEEE Trans. Emerg. Topics Comput. Intell.* **8**(3), 2269–2282 (2024). <https://doi.org/10.1109/TETCI.2024.3369407>
22. D. O'Neill, H. Al-Sahaf, B. Xue, M. Zhang, Common subtrees in related problems: A novel transfer learning approach for genetic programming. In: *2017 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1287–1294 (2017). <https://doi.org/10.1109/CEC.2017.7969453>
23. J. Russell, N. Pillay, A selection hyper-heuristic for transfer learning in genetic programming. In: *Proceedings of the Companion Conference on Genetic and Evolutionary Computation. GECCO '23 Companion*, Association for Computing Machinery, New York, NY, USA, pp. 631–634. (2023). <https://doi.org/10.1145/3583133.3590686>
24. N. Pillay, Transfer learning in evolutionary spaces. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion. GECCO '24 Companion*, Association for Computing Machinery, New York, NY, USA, pp. 1005–1016. (2024). <https://doi.org/10.1145/3638530.3648431>

25. Y. Liu, G. Taylor, Z. Huang, Novel application of mutual information in transfer learning for genetic programming. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion. GECCO '25 Companion, Association for Computing Machinery, New York, NY, USA, pp. 635–638. (2025). <https://doi.org/10.1145/3712255.3726607>
26. N. Pillay, Evolving hyper-heuristics for a highly constrained examination timetabling problem. In: Proceedings of the 8th International Conference on the Practice and Theory of Automated Timetabling (PATAT'10), pp. 336–346 (2010)
27. N.R. Sabar, M. Ayob, R. Qu, G. Kendall, A graph coloring constructive hyper-heuristic for examination timetabling problems. *Appl. Intell.* **37**, 1–11 (2012)
28. A. Soghier, R. Qu, Adaptive selection of heuristics for assigning time slots and rooms in exam timetables. *Appl. Intell.* **39**, 438–450 (2013)
29. S.A. Rahman, A. Bargiela, E.K. Burke, E. Özcan, B. McCollum, P. McMullan, Adaptive linear combination of heuristic orderings in constructing examination timetables. *Eur. J. Oper. Res.* **232**(2), 287–297 (2014)
30. T. Müller, Itc 2007 solver description: a hybrid approach. *Ann. Oper. Res.* **172**(1), 429–446 (2009)
31. M. Atsuta, K. Nonobe, T. Ibaraki, Itc2007 track 2, an approach using general csp solver. Practice and theory of automated timetabling (PATAT 2008), p. 19–22 (2008)
32. C. Gogos, G. Goulas, P. Alefragis, V. Kolonias, E. Housos, Distributed scatter search for the examination timetabling problem. *PATAT* **2010**, 211 (2010)
33. Y. Bykov, S. Petrovic, A step counting hill climbing algorithm applied to university examination timetabling. *J. Sched.* **19**, 479–492 (2016)
34. C. Rajah, Structure based partial solution search for the examination timetabling problem. PhD thesis, School of Mathematics, Statistics and Computer Science. University of KwaZulu-Natal (2021)
35. D. Van Bulck, D. Goossens, A. Schaefer, Multi-neighbourhood simulated annealing for the itc-2007 capacitated examination timetabling problem. *J. Sched.* **28**(2), 217–232 (2025). <https://doi.org/10.1007/s10951-023-00799-1>
36. B.H. Korte, J. Vygen, *Combinatorial Optimization* vol. 1, (Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2011)
37. P. Ross, J.G. Marín-Blázquez, S. Schulenburg, E. Hart, Learning a procedure that can solve hard bin-packing problems: a new ga-based approach to hyper-heuristics. In: Genetic and Evolutionary Computation—GECCO 2003: Genetic and Evolutionary Computation Conference Chicago, IL, USA, July 12–16, 2003 Proceedings, Part II, pp. 1295–1306, Springer (2003)
38. N. Pillay, A study of evolutionary algorithm selection hyper-heuristics for the one-dimensional bin-packing problem. *S. Afr. Comput. J.* **48**(1), 31–40 (2012)
39. K. Sim, E. Hart, B. Paechter, Learning to solve bin packing problems with an immune inspired hyper-heuristic. Note: paper presented at the 12th European Conference on Artificial Life held in Taormina, Italy on 2-6 September 2013 This work was funded by EPSRC grant P/J1021628/1 Real World Optimisation with Life-Long Learning School: iidi (2024), <https://doi.org/10.7551/978-0-262-31709-2-c-h126>. <http://researchrepository.napier.ac.uk/id/eprint/6251>
40. K. Sim, E. Hart, An improved immune inspired hyper-heuristic for combinatorial optimisation problems. In: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, pp. 121–128 (2014)
41. E. Singh, N. Pillay, A study of ant-based pheromone spaces for generation constructive hyper-heuristics. *Swarm Evol. Comput.* **72**, 101095 (2022)
42. A.C. Alvim, C.C. Ribeiro, F. Glover, D.J. Aloise, A hybrid improvement heuristic for the one-dimensional bin packing problem. *J. Heuristics* **10**, 205–229 (2004)
43. K. Fleszar, C. Charalambous, Average-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem. *Eur. J. Oper. Res.* **210**(2), 176–184 (2011)
44. T. Dokeroglu, A. Cosar, Optimization of one-dimensional bin packing problem with island parallel grouping genetic algorithms. *Comput. Ind. Eng.* **75**, 176–186 (2014)
45. M. Quiroz-Castellanos, L. Cruz-Reyes, J. Torres-Jimenez, C. Gómez, H.J.F. Huacuja, A.C. Alvim, A grouping genetic algorithm with controlled gene transmission for the bin packing problem. *Comput. Oper. Res.* **55**, 52–64 (2015)
46. T. Kucukyilmaz, H.E. Kiziloz, Cooperative parallel grouping genetic algorithm for the one-dimensional bin packing problem. *Comput. Ind. Eng.* **125**, 157–170 (2018)
47. A.C. Silva, C.C.H. Borges, An improved heuristic based genetic algorithm for bin packing problem. In: 2019 8th Brazilian Conference on Intelligent Systems (BRACIS), (IEEE 2019), pp. 60–65

48. É. Taillard, Parallel iterative search methods for vehicle routing problems. *Netw.* **23**(8), 661–673 (1993)
49. T. Volgenant, R. Jonker, The symmetric traveling salesman problem and edge exchanges in minimal 1-trees. *Eur. J. Oper. Res.* **12**(4), 394–403 (1983)
50. Y. Rochat, É.D. Taillard, Probabilistic diversification and intensification in local search for vehicle routing. *J. Heuristics.* **1**, 147–167 (1995)
51. J. Xu, J.P. Kelly, A network flow-based tabu search heuristic for the vehicle routing problem. *Transp. Sci.* **30**(4), 379–393 (1996)
52. S. Lin, Computer solutions of the traveling salesman problem. *Bell Syst. Tech. J.* **44**(10), 2245–2269 (1965)
53. P. Toth, D. Vigo, The granular tabu search and its application to the vehicle-routing problem. *Inform. J. Comput.* **15**(4), 333–346 (2003)
54. E.K. Burke, E.K. Burke, G. Kendall, G. Kendall, *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques* (Springer, Boston, MA, 2014)
55. L. Di Gaspero, B. McCollum, A. Schaerf, The second international timetabling competition (itc-2007): curriculum-based course timetabling (track 3). Technical report, Technical Report QUB/IEEE/Tech/ITC2007/CurriculumCTT/v1.0 (2007)
56. A. Scholl, R. Klein, C. Jürgens, Bison: a fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Comput. Oper. Res.* **24**(7), 627–645 (1997). [https://doi.org/10.1016/S0305-0548\(96\)00082-2](https://doi.org/10.1016/S0305-0548(96)00082-2)
57. E. Falkenauer, A hybrid grouping genetic algorithm for bin packing. *J. heuristics.* **2**, 5–30 (1996)
58. N. Christofides, J.E. Beasley, The period routing problem. *Netw.* **14**(2), 237–256 (1984)
59. N. Pillay, R. Qu, Assessing hyper-heuristic performance. *J. Oper. Res. Soc.* **72**(11), 2503–2516 (2021)
60. H.B. Mann, D.R. Whitney, On a test of whether one of two random variables is stochastically larger than the other. *Ann. Math. Stat.* **18**(1), 50–60 (1947). <https://doi.org/10.1214/aoms/1177730491>
61. P.A. Djang, P.R. Finch, Solving one dimensional bin packing problems. *J. Heuristics.* 123–144 (1998)
62. P. Ross, S. Schulenburg, J.G. Marín-Blázquez, E. Hart, Hyper-heuristics: learning to combine simple heuristics in bin-packing problems. In: *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation*, pp. 942–948 (2002)
63. K. Sim, E. Hart, B. Paechter, A hyper-heuristic classifier for one dimensional bin packing problems: improving classification accuracy by attribute evolution. In: *Parallel Problem Solving from Nature-PPSN XII: 12th International Conference, Taormina, Italy, September 1-5, 2012, Proceedings, Part II* 12, pp. 348–357 (2012). Springer
64. P. Garrido, C. Castro, Stable solving of cvrps using hyperheuristics. In: *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation*, pp. 255–262 (2009)
65. R. Qu, E.K. Burke, B. McCollum, Adaptive automated construction of hybrid heuristics for exam timetabling and graph colouring problems. *Eur. J. Oper. Res.* **198**(2), 392–404 (2009)
66. P. Kilby, P. Prosser, P. Shaw, Dynamic vrps: a study of scenarios. *Univ. Strathclyde Tech. Rep.* **1**(11) (1998)
67. G. Clarke, J.W. Wright, Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.* **12**(4), 568–581 (1964)
68. R. Mole, S. Jameson, A sequential route-building algorithm employing a generalised savings criterion. *J. Oper. Res. Soc.* **27**(2), 503–511 (1976)
69. B.E. Gillett, L.R. Miller, A heuristic algorithm for the vehicle-dispatch problem. *Oper. Res.* **22**(2), 340–349 (1974)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.